

The Microsoft Foundry Agent Development Guide

Industrializing AI Agents from Prototype to Production

June 2026

Contents

1	Overview	4
1.1	Build 2026 refresh (June 2026)	5
1.2	Preview vs GA — read this before committing	6
2	The Industrialization Journey at a Glance	8
3	Foundry Platform & Agent Types Decoded	8
3.1	The three agent types — pick one consciously	8
3.2	The tool & knowledge ecosystem	9
3.3	The unified project plane (what you get for free)	10
4	Build Locally with Microsoft Agent Framework v1.0	10
4.1	What's stable in v1.0	11
4.2	Hello agent (Python)	11
4.3	Foundry Toolkit for VS Code	11
4.4	Other supported authoring paths	12
5	Agent Harness — Shell, Files, and Long Sessions	12
5.1	2.1 Local Shell Harness with approval flows	12
5.2	2.2 Hosted Shell Harness — same code, sandbox runtime	13
5.3	2.3 Context Compaction for long-running sessions	13
5.4	2.4 Multi-agent composition with the GitHub Copilot SDK	14
6	Make Agents Stateful — Memory in Foundry	14
6.1	What's new in this refresh	15
6.2	Pricing (consumption begins June 1, 2026)	15
6.3	Memory governance — what to store, when to forget	15
7	Toolbox in Foundry — One Endpoint, Any Agent	16
7.1	The problem Toolbox solves	16
7.2	The four pillars	17
7.3	Anatomy	18
7.4	Endpoint shape	19
7.5	Build a toolbox (Python)	19
7.6	Consume from any framework	19
7.7	Promote to default	20
7.8	Tool governance — fillable spec template	20
8	Hosted Agents in Foundry Agent Service	21
8.1	Why traditional compute doesn't fit agents	22
8.2	Core capabilities (2026 refresh)	22
8.3	Container model	22
8.4	Identities involved	23
8.5	Sessions vs Conversations	23
8.6	Choosing a protocol	24
8.7	Versioning, canary, and blue/green	25
8.8	One-command deployment	25

9 End-to-End Reference Architecture	25
9.1 Layer responsibilities	26
10 Industrialization with azd and CI/CD	27
10.1 Recommended project structure	27
10.2 Key pieces of agent.yaml	28
10.3 Promotion workflow (GitHub Actions)	28
10.4 One-command developer flow	28
11 Security & Governance	29
11.1 Trust & identity boundaries — at a glance	30
11.2 Identity & access (no shared service accounts)	30
11.3 Network isolation — what's in, what's still public	31
11.4 Content safety & DLP — what is built in vs what you must configure	31
11.5 Secrets	31
11.6 Third-party tool risk	32
11.7 Defending against LLM-specific attacks	32
11.8 Per-agent least privilege — concrete pattern	32
11.9 Audit trail per tool call	33
12 Observability & Operations	33
12.1 What's in scope	34
12.2 Recommended baseline alerts	34
13 Cost & FinOps	34
13.1 Memory pricing reminder (effective June 1, 2026)	35
13.2 Patterns to adopt early	36
14 Decision Matrix — Hosted Agent vs Prompt-Based Agent	36
14.1 When <i>not</i> to build an agent at all	36
14.2 Hosted Agent vs Prompt-Based Agent	37
15 Test & Simulation	37
15.1 Inject failures at the protocol layer	37
15.2 Validate canary metrics with KQL	38
15.3 Adversarial coverage	38
15.4 Chaos for downstream services	38
16 Anti-Patterns to Avoid	38
16.1 Architecture anti-patterns	38
16.2 Tooling anti-patterns	39
16.3 Data & memory anti-patterns	39
16.4 Process & governance anti-patterns	40
17 Production Readiness Checklist	40
17.1 Architecture	41
17.2 Identity & Security	41
17.3 Tools	41
17.4 State	41

17.5 Operations	41
17.6 Distribution	41
17.7 Verify-before-go-live (must be green before traffic)	41
18 Required Personas for Industrialization	42
19 Service Limits & Quotas	43
19.1 Documented limits (verify on the quotas page)	43
19.2 Planning rules-of-thumb (architectural assumptions, not contractual)	44
19.3 Model quota & shared-quota pools	45
19.4 Quota-increase workflow	45
20 Regional Availability & SLA	45
20.1 Region selection checklist	46
20.2 SLA reference (preview vs GA)	46
21 Hosted Agent Execution Pricing Detail	47
21.1 Pricing changes frequently — verify before committing	47
21.2 Hosted Agent compute (preview pricing — verify on the live page)	47
21.3 Memory pricing — preview, billing scheduled to start June 1, 2026	47
21.4 Worked example — a typical “support triage” agent	48
21.5 PTU vs PAYG — the deployment-mode trade-off	48
22 Fine-Tuning & Custom Models	49
22.1 When fine-tuning beats prompting + RAG	49
22.2 How to do it on Foundry	49
22.3 Choosing a model — workload fit, not leaderboard rank	49
22.4 Fireworks AI on Foundry (GA)	50
23 Enterprise Context — Work IQ, Fabric IQ, Foundry IQ	51
23.1 Foundry IQ at Build 2026 (June 2026)	51
23.2 Foundry IQ — curated knowledge & multi-source agentic retrieval	52
23.3 Fabric IQ — business data via Microsoft Fabric	52
23.4 Work IQ — Microsoft 365 graph (mail, files, chats, meetings)	53
23.5 Pattern: chain Foundry IQ → Fabric IQ → Work IQ in a single agent	53
24 Compliance, Data Residency & Sovereignty	54
25 Advanced Quality Evaluation — Coherence, Groundedness, Hallucination	55
25.1 Evaluate in the Foundry portal — four targets	55
25.2 Built-in evaluators (Azure AI Evaluation SDK)	55
25.3 Example — wire a continuous eval into CI	56
25.4 Red Team Agent (preview)	57
25.5 Build your golden dataset from production traces	57
26 Migration Paths & .NET Parity	57
26.1 From Semantic Kernel to MAF v1.0	57
26.2 From AutoGen to MAF v1.0	57
26.3 .NET parity	57

26.4 Update / lifecycle policy	58
27 Lifecycle Management — Versioning, Rollback, Decommissioning	58
27.1 Versioning model	58
27.2 Rollouts and rollback	59
27.3 Continuous improvement loop	59
27.4 Model upgrade strategy	59
27.5 Decommissioning checklist	60
28 Agent 365 & Enterprise Control Plane	60
28.1 What Agent 365 provides	60
28.2 Governance with Azure Policy	61
28.3 Responsible AI gating	61
28.4 Adoption pattern	62
29 References & Further Reading	62
29.1 Get started	62
29.2 Source articles (this guide synthesizes and extends them)	62
29.3 Microsoft Learn	62
29.4 Tooling	63
29.5 Build 2026 sessions & skilling	63
29.6 Community	63

Industrializing AI agents on Microsoft Foundry — from local prototype to production-grade enterprise deployment.

1 Overview

What you will learn

- The six Foundry building blocks the rest of the guide builds on (MAF, Toolkit, Toolbox, Memory, Hosted Agents, Observability) and the GA / Preview status of each as of June 2026 (post-Build 2026).
- Why this guide is positioned as a single entry point for industrialization rather than a per-feature reference.
- The disclaimer that frames every preview-marked claim in this guide.

Building a working agent on a developer laptop is no longer the hard part. The hard part is taking that same agent and operating it for **thousands of users**, with **per-session isolation**, **enterprise identity**, **governed tools**, **persistent memory**, **observability**, and **scale-to-zero economics** — without rebuilding the platform every time.

Microsoft Foundry now provides an end-to-end set of services for that journey, structured around six capabilities that landed (or stabilized) together:

Capability	Status	What it changes
Microsoft Agent Framework v1.0	GA	Stable Python + .NET runtime that consolidates patterns from Semantic Kernel and AutoGen, with documented migration paths from each
Foundry Toolkit for VS Code	GA	Local create / debug / deploy loop with traces
Memory in Foundry Agent Service	Public Preview	Managed long-term memory, no DB to provision
Toolbox in Foundry	Public Preview	One MCP endpoint exposing many tools, governed centrally
Hosted Agents in Foundry Agent Service (refresh)	Public Preview (June 2026)	Per-session VM-isolated sandbox, scale-to-zero, persistent FS, per-agent Entra ID. The Foundry Agent Service control plane is GA, but the hosted-agents compute / runtime is still in public preview — see the official Hosted Agents (preview) page
Observability in Foundry Control Plane	Core capabilities GA, advanced features still rolling out	End-to-end OpenTelemetry tracing and continuous evaluation are GA; Red Teaming Agent and some advanced eval flows remain Public Preview

This guide walks through the full industrialization pipeline, with code samples, an end-to-end reference architecture, governance and FinOps controls, and a production checklist.

1.1 Build 2026 refresh (June 2026)

Microsoft Build 2026 extended the agent stack the rest of this guide builds on. The headline changes folded into this revision:

What's new	Status	Why it matters
Foundry IQ knowledge bases + the Foundry IQ MCP server	GA	SLA-backed, multi-source agentic retrieval exposed over open MCP — callable from MAF, Claude, ChatGPT, LangChain (see Knowledge & IQ)

What's new	Status	Why it matters
Foundry IQ Serverless (Developer tier)	Public Preview	Scale-to-zero RAG billed in Compute Units; no clusters to provision
New Foundry IQ knowledge sources — Work IQ, Fabric IQ (Data agents + Ontology), File Search, Azure SQL, MCP	Public Preview	One knowledge base spans enterprise + structured data, no custom connectors
Microsoft Web IQ in Foundry IQ	Limited access	Sub-165 ms, zero-data-retention web grounding that honors publisher preferences
Fireworks AI on Foundry	GA	Production-grade open-model inference (DeepSeek, Qwen, Kimi, GLM, Gemma, Llama...) + BYOM through one Azure endpoint with enterprise SLAs
Model Router + instant models	GA / Preview	Auto-route each request by cost/latency/quality; use deployment-less models without provisioning
Agentic retrieval quality	GA	Answer quality up to +20% , recall up to +54% vs single-shot RAG, with server-side token caching

The model-operations discipline that frames these additions — **Select** → **Evaluate** → **Optimize** → **Operate** → **Improve** — is summarized in Cost & FinOps and Advanced Quality Evaluation.

1.2 Preview vs GA — read this before committing

Microsoft Foundry's agent stack is in **rapid iteration in 2026**. Several capabilities used in this guide are **public preview** and **must not be used to back external production SLAs** until they reach GA:

Capability	Status today	Implication
Microsoft Agent Framework v1.0 (SDK)	GA	Safe for production code

Capability	Status today	Implication
Foundry Toolkit for VS Code	GA	Safe for inner-loop tooling
Foundry Agent Service control plane	GA	Subject to a Microsoft SLA — verify the exact figure in the Azure SLA / Product Terms before quoting it externally
Hosted Agents compute (per-session sandbox)	Public preview	No SLA — the Foundry Agent Service control plane is GA, but the hosted-agents compute is still in public preview. Do not back external production SLAs with hosted-agents until they reach GA
Foundry Toolbox	Public preview	No SLA; tool catalogue evolving
Foundry Memory	Public preview	No SLA; billing scheduled to begin June 1, 2026 — verify before production go-live
AI Red Teaming Agent Foundry IQ knowledge bases + Foundry IQ MCP server	Public preview GA	No SLA SLA-backed retrieval layer; safe for production grounding. Some sources/security features remain preview (see table below)
Foundry IQ Serverless (Developer tier)	Public preview	No SLA; Compute-Unit billing expected to start late 2026 — free until then
Fireworks AI on Foundry	GA	Enterprise SLAs; excluded from EU Data Boundary , no FedRAMP/PCI — read the data-privacy caveats before regulated use

All preview prices, quotas and feature lists in this guide are **planning assumptions**. Always cross-check the live Microsoft Learn page before a commercial commitment — links are in the References section.

2 The Industrialization Journey at a Glance

What you will learn

- The five-step path from a local prototype to a hosted, channel-published agent.
- Which artifact (image + `agent.yaml`) ties the steps together so each step stays reversible.

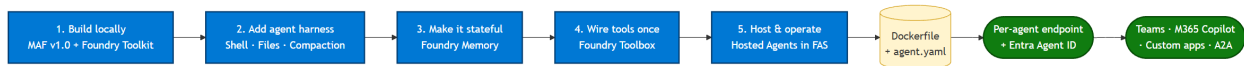


Figure 1: Diagram 1

Each step is **independent and reversible** — you can adopt them in any order — but they reinforce each other and are designed around a single deployment artifact: a container image plus an `agent.yaml` declaration deployable in one `azd deploy`.

3 Foundry Platform & Agent Types Decoded

What you will learn

- The three agent flavours (Prompt-based, Workflow, Hosted Agent) and where each one fits.
- What a Foundry project gives you out of the box (RBAC, connections, tracing, evaluations).
- The three external connector surfaces — Foundry IQ, Logic Apps, MCP server catalog — and why any agent type can use any of them.

Before the deep-dives, anchor the mental model: **Microsoft Foundry is a single project-scoped workspace** that unifies models, tools, agents, knowledge, identity, and observability behind one RBAC plane. An agent is never a stand-alone artifact — it always lives inside a Foundry **project**, inherits the project's connections (model deployments, Azure AI Search indexes, Storage, Key Vault), and is governed by the project's RBAC and policies.

3.1 The three agent types — pick one consciously

Foundry exposes **three agent flavours** with different authoring surfaces, runtime models and trade-offs. Most enterprises end up using all three for different use-cases.

Type	Authoring	Runtime	When to choose
Prompt-based agent (declarative)	Foundry portal Agent Playground or <code>agent.yaml</code>	Foundry-managed serverless (no container)	Fast start; chat-style agents; minimal custom code; no per-session filesystem needed

Type	Authoring	Runtime	When to choose
Workflow agent (multi-step)	Foundry portal designer or MAF Workflow (declarative or programmatic)	Foundry-managed serverless with deterministic step graph	Multi-step orchestrations with conditional branches, parallel fan-out, hand-offs; observable per step
Hosted Agent (BYO container)	Dockerfile + agent.yaml (azd deploy)	Per-session VM-isolated sandbox , scale-to-zero, persistent FS	Custom code, custom harness (MAF / LangGraph / Copilot SDK / your own), need shell + filesystem, BYO VNet

§14 Decision Matrix expands this with concrete capability comparisons.

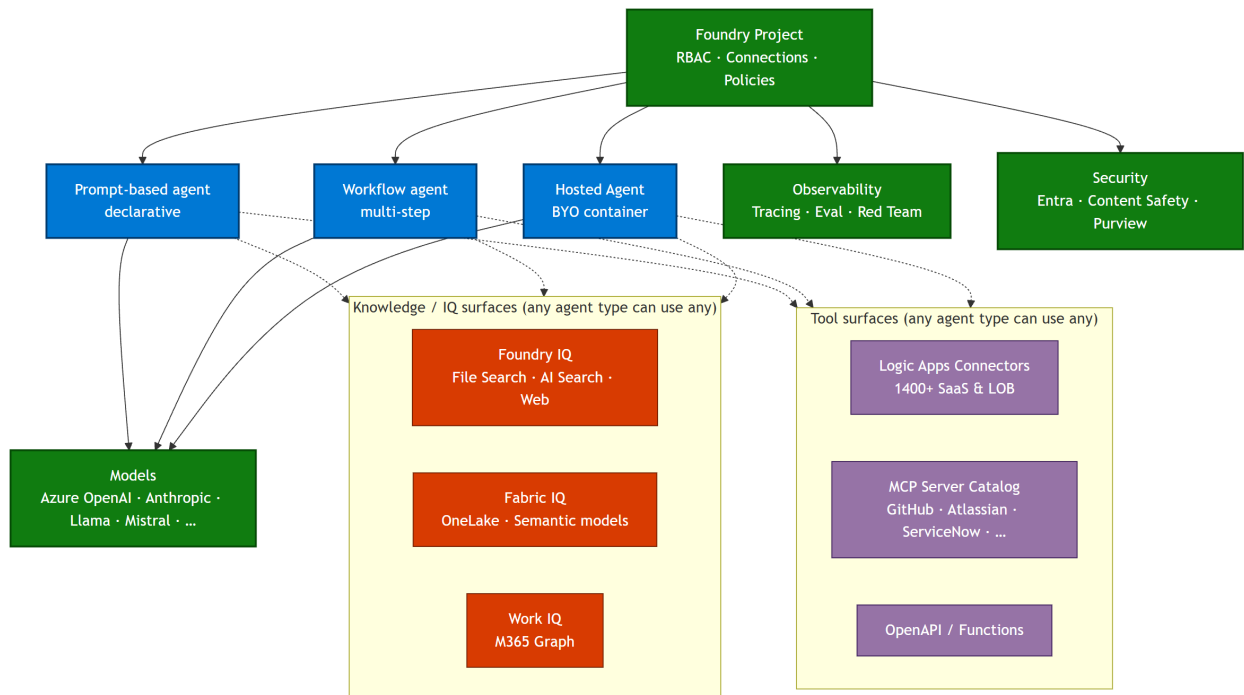


Figure 2: Diagram 2

3.2 The tool & knowledge ecosystem

A Foundry agent reaches outside through **three connector surfaces**, all callable directly or — recommended — through a Toolbox (§7) for reuse and governance:

- **Foundry IQ** — the managed enterprise-knowledge layer. Knowledge bases and the **Foundry IQ MCP server** are **GA** (Build 2026): one multi-source knowledge base spans Azure AI Search, Azure Blob, OneLake, **Work IQ**, **Fabric IQ** (Data agents + Ontology), **File Search**, **Azure SQL**

and **MCP**, with agentic retrieval, document-level security and cross-source ranking. Foundry-managed primitives (`file_search`, `azure_ai_search`, `web_search`) and **Microsoft Web IQ** (low-latency, zero-retention web grounding) are reachable the same way. **Foundry IQ Serverless** (Developer tier, preview) adds scale-to-zero RAG. See Enterprise Context.

- **Logic Apps connectors** — **1,400+** Microsoft-curated SaaS & LOB connectors (Salesforce, SAP, ServiceNow, Workday, Dynamics, Office 365, GitHub, Jira, Confluence, ...) exposed natively to agents with no custom code.
- **MCP server catalog** — open Model Context Protocol servers (GitHub Copilot MCP, Atlassian, Notion, your own) callable through Toolbox with OAuth pass-through or MI.
- **OpenAPI / Azure Functions / A2A** — wrap any HTTP API or another agent as a tool.

3.3 The unified project plane (what you get for free)

Inside one Foundry project, **the platform itself supplies**:

Capability	Built-in mechanism
RBAC	Azure RBAC + Entra Agent ID — same model as any Azure resource
Connections	Centrally configured (Storage, Key Vault, AI Search, Logic Apps, MCP) — agents reference by name
Tracing	OpenTelemetry auto-instrumentation across model · tool · agent calls
Evaluations	Built-in evaluators + AI Red Teaming Agent
Content Safety	Prompt Shields, jailbreak detection, harmful content filtering at the project level
Cost & quota	Per-project token budgets, Azure Cost Management integration
Lifecycle	Agent versioning, weighted rollouts, stable endpoints

Implication for the rest of this guide: every code sample below assumes you have one Foundry project with at least one model deployment and one connection (Storage or Key Vault). All §-numbered sections build on this foundation.

4 Build Locally with Microsoft Agent Framework v1.0

What you will learn

- What is GA in MAF v1.0 and the documented migration paths from Semantic Kernel and AutoGen.
- The minimum Python sample to instantiate an agent, plus the .NET equivalent.
- Which authoring runtimes (MAF, LangChain, LlamaIndex, custom) are first-class on Foundry.

4.1 What's stable in v1.0

Microsoft Agent Framework (MAF) is the open-source SDK and runtime that unifies the enterprise foundations of **Semantic Kernel** with the multi-agent orchestration patterns of **AutoGen**. Migration guides are published for both predecessors so existing investments carry forward.

Pillar	What you get
Multi-model	Azure OpenAI, Anthropic, Google Gemini, Amazon Bedrock, Fireworks AI (GA — DeepSeek, Qwen, Kimi, GLM, Gemma, Llama + BYOM), Ollama, OpenAI direct, and more — plus Model Router to pick per request
Workflows	Programmatic <i>and</i> declarative multi-step pipelines, visualizable in DevUI
Native Foundry integration	Memory, hosted agents, Observability, and Foundry Tools as first-class building blocks
Open standards	MCP, A2A, OpenAPI built in
Languages	Python and .NET, identical concepts

4.2 Hello agent (Python)

```
import asyncio
from agent_framework.azure import AzureOpenAIResponsesClient
from azure.identity import AzureCliCredential

async def main():
    agent = AzureOpenAIResponsesClient(
        credential=AzureCliCredential(),
    ).as_agent(
        name="SupportTriageBot",
        instructions="You are an expert support triage agent.",
    )
    response = await agent.run("Analyze this ticket and classify its
    ↪ priority.")
    print(response.text)

asyncio.run(main())
```

4.3 Foundry Toolkit for VS Code

The Toolkit (formerly *AI Toolkit for VS Code*) is now **GA** and gives you:

- Project scaffolding from templates or from GitHub Copilot prompts
- Local run + step-through debugging with **trace visualization**
- Direct Deploy to Foundry Agent Service action — no leaving VS Code
- Pre-release channel for the newest Hosted Agents experience

This is the recommended inner loop for any agent destined for Foundry hosting.

4.4 Other supported authoring paths

You are **not locked into MAF**. Foundry meets developers where they are:

Path	When	Notes
Agent Playground (Foundry portal UI)	POC, business-author “no-code” agents, prompt iteration	Declarative; outputs an <code>agent.yaml</code> you can promote to a hosted agent later
Microsoft Agent Framework v1.0	Production custom agents in Python or .NET	First-class Foundry integration; this guide’s default
LangChain / LangGraph	Existing LangChain investments	Use the Foundry Inference SDK + Toolbox MCP client; observability still flows through OpenTelemetry
LlamaIndex	RAG-first agents already on LlamaIndex	Foundry connections (AI Search, <code>file_search</code>) usable through standard LlamaIndex retrievers
Semantic Kernel	Migrating apps	First-class migration path → MAF (see §26 Migration)
AutoGen	Multi-agent research code	First-class migration path → MAF (see §26 Migration)
Custom / no framework	Special runtime constraints	Hosted Agent + your own HTTP loop; Toolbox still callable via MCP HTTP

The constant across all paths is the **Foundry project + agent.yaml + Entra Agent ID** triple — the framework above the `azd deploy` line is your choice.

5 Agent Harness — Shell, Files, and Long Sessions

What you will learn

- How to wire shell, file, and long-session compaction patterns into a MAF agent.
- The single-line change that moves shell execution from local to a Hosted Agent sandbox.
- Why context compaction is not optional for sessions that run over hours.

Orchestrating *multiple* agents is half the picture; the other half is what one agent does autonomously over a long task: run shell commands, read/write files, manage context across hours of work without losing coherence. MAF ships three foundational harness patterns.

5.1 2.1 Local Shell Harness with approval flows

```
@tool(approval_mode="always_require")
def run_bash(command: str) -> str:
    """Execute a shell command and return the result."""
    result = subprocess.run(command, shell=True, capture_output=True,
    ↪ text=True)
    return result.stdout

local_shell_tool = client.get_shell_tool(func=run_bash)
agent = Agent(
    client=client,
    instructions="You are a helpful assistant that can run shell commands.",
    tools=[local_shell_tool],
)
```

□ Always run local shell execution in an isolated environment with approval enabled.

5.2 2.2 Hosted Shell Harness — same code, sandbox runtime

```
shell_tool = client.get_shell_tool() # execution runs in provider-managed
    ↪ sandbox
agent = Agent(client=client, instructions="...", tools=shell_tool)
```

A **single-line change** moves command execution from the laptop to the VM-isolated sandbox that powers Hosted Agents — no Dockerfile changes, no infrastructure rewiring.

5.3 2.3 Context Compaction for long-running sessions

```
from agent_framework import (
    Agent, InMemoryHistoryProvider,
    CompactionProvider, SlidingWindowStrategy,
)

agent = Agent(
    client=client,
    instructions="You are a helpful assistant.",
    tools=[get_weather],
    context_providers=[
        InMemoryHistoryProvider(),
        CompactionProvider(
            before_strategy=SlidingWindowStrategy(keep_last_groups=3)
        ),
    ],
)
```

Keeps the agent inside its token budget while preserving the most recent meaningful conversation groups — essential for sessions that run for hours or days.

5.4 2.4 Multi-agent composition with the GitHub Copilot SDK

MAF acts as the **orchestration backbone** while delegating heavy code-related work (MCP tools, shell, file ops) to a Copilot-SDK-powered sub-agent. Sequential, concurrent, hand-off, and coding agent patterns are all supported, with checkpointing, middleware pipelines, and human-in-the-loop control built into the execution layer.

6 Make Agents Stateful — Memory in Foundry

What you will learn

- How to attach Foundry Memory to an agent via FoundryMemoryProvider and what scope to pin.
- Memory's per-event / per-memory / per-retrieval pricing model (free during pre-view, billed from June 2026).
- The governance rules — what to scope, what to never store, how to honor right-to-erasure.

Status: Public Preview · **Production readiness:** Use with caution — no SLA, billing scheduled to begin **June 1, 2026**, schema and pricing can still evolve. Treat any sensitive data carefully and keep an exit plan.

Production agents need to remember user preferences, prior context, and domain facts across sessions — without forcing every team to provision their own database.

```
from agent_framework import InMemoryHistoryProvider
from agent_framework.azure import AzureOpenAIResponsesClient,
    ↪ FoundryMemoryProvider
from azure.ai.projects import AIProjectClient
from azure.identity import AzureCliCredential
import os

project_client = AIProjectClient(
    endpoint=os.environ["AZURE_AI_PROJECT_ENDPOINT"],
    credential=AzureCliCredential(),
)

agent = AzureOpenAIResponsesClient(
    credential=AzureCliCredential(),
).as_agent(
    name="CustomerSuccessAgent",
    instructions="You are a proactive customer success agent.",
    context_providers=[
        InMemoryHistoryProvider(),
        FoundryMemoryProvider(
            project_client=project_client,
            memory_store_name=memory_store.name,
            scope="user_123", # segments memory per user
```

```
)  
  ],  
)
```

6.1 What's new in this refresh

Capability	Why it matters
Native MAF & LangGraph integration	No middleware glue — memory works out of the box in either framework
Memory item CRUD API	Inspect, edit, delete specific facts the agent stored — full transparency for users and DPO requests
Custom user scope header	Segment memory by your own userId, not necessarily Entra — works with any IAM system

6.2 Pricing (consumption begins June 1, 2026)

Component	Price
Short-term memory	\$0.25 per 1K events stored
Long-term memory	\$0.25 per 1K memories per month
Memory retrieval	\$0.50 per 1K retrievals

Memory is **free during preview** before billing begins.

6.3 Memory governance — what to store, when to forget

Memory is a **risk surface disguised as a feature**. Treat it like a small database that needs a data-protection policy from day one.

Decision	Recommended default
What MAY be memorized	User preferences, declared profile facts, ongoing task state, factual answers the user confirmed
What must NEVER be memorized	Secrets, credentials, payment / health data unless explicitly authorized; PII covered by stricter regulation; "facts" the agent inferred but the user did not confirm
Scope strategy	Distinct scope per user , per tenant , per workspace — never a global scope for user-derived content

Decision	Recommended default
Retention	Set a TTL per memory class (e.g. session-context: 7 d; preferences: 12 mo; task-state: until task closed). Run a periodic cleanup job
User rights (GDPR / DPO)	Expose read and delete via the Memory CRUD API; document the SLA for “right to erasure” requests
Export	Provide a <code>memory.export(scope=...)</code> operation — useful for portability and audit
Memory poisoning	Validate any new fact before persistence (model-as-judge, optional human review for sensitive scopes); reject contradictions to canonical sources
Conflict between memory and source-of-truth	Memory is never authoritative — when a database / system-of-record disagrees, the agent must trust the source-of-truth and update memory
Periodic validation	Schedule a recurring job that re-validates long-term memories against system-of-record sources; flag stale or contradicted facts
Tenant separation	Hard fail any retrieval crossing tenant boundaries — write a contract test for it

Operational rule of thumb: “If you cannot explain to a user — in one sentence — *why* the agent remembers this, *for how long*, and *how they can delete it*, the agent should not be storing it.”

7 Toolbox in Foundry — One Endpoint, Any Agent

What you will learn

- Why a Toolbox replaces N×M tool-to-agent integration and where it sits in the call chain.
- How to build, version, promote, and consume a Toolbox from any MCP-capable runtime.
- The 15-field tool governance template every production-grade tool should pass.

Status: Public Preview · **Production readiness:** Recommended for new builds; expect API surface to evolve before GA. Discover and Govern pillars are not fully shipped — design with that in mind.

7.1 The problem Toolbox solves

A single onboarding agent might call five tools (Entra account creation, GitHub repo access, cloud provisioning, ADO tasks, Teams welcome). That means **five auth models, five owning teams, five**

integration codebases. Multiply by every agent in the org and you get duplicated credentials, inconsistent governance, and zero discoverability.

7.2 The four pillars

Pillar	Available	What you get
Discover	Coming soon	Find approved tools without browsing catalogs
Build	Today	Curate tools into a named, versioned, reusable bundle
Consume	Today	One MCP-compatible endpoint for the whole bundle
Govern	Coming soon	Centralized auth + observability across every tool call

7.3 Anatomy

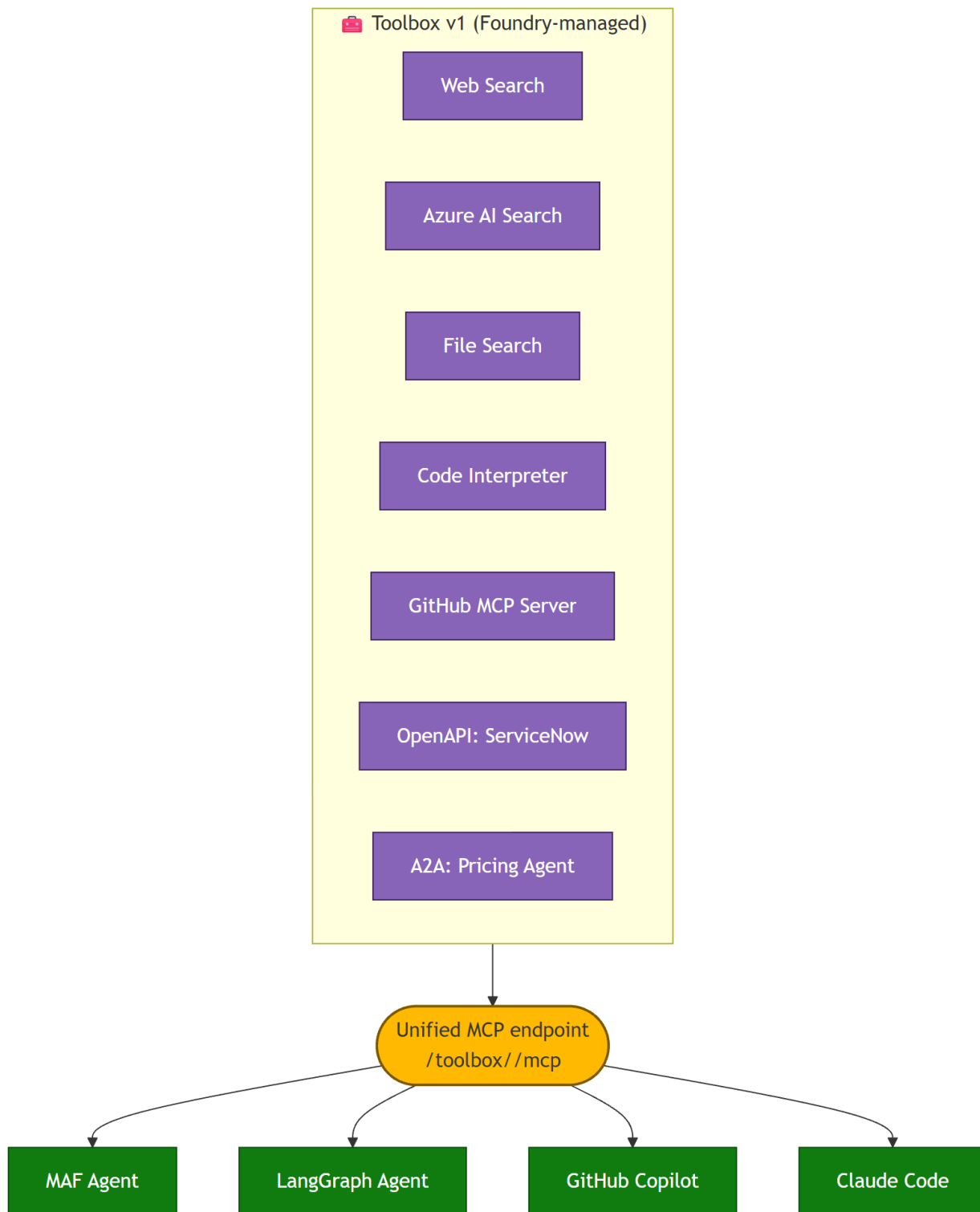


Figure 3: Diagram 3

7.4 Endpoint shape

`https://<resource>.services.ai.azure.com/api/projects/<project>/toolbox/<toolbox-name>/mcp?api-version=v1`

A toolbox is **versioned**: you can iterate freely on a v2 while consumers stay pinned to v1. Promote a version to be the default once it's production-ready, and existing consumers automatically pick it up at the unversioned endpoint.

7.5 Build a toolbox (Python)

```
toolbox_version = client.beta.toolboxes.create_toolbox_version(
    toolbox_name="customer-feedback-triaging-toolbox",
    description="Search docs and triage feedback; act in GitHub.",
    tools=[
        {
            "type": "web_search",
            "description": "Retrieve public docs on official websites",
            "custom_search_configuration": {
                "project_connection_id": "<BING_CONNECTION>",
                "instance_name": "<BING_INSTANCE>"
            }
        },
        {
            "type": "azure_ai_search",
            "name": "product-manuals-ai-search",
            "description": "Search internal product documentation",
            "azure_ai_search": {
                "indexes": [
                    {
                        "index_name": "<INDEX>",
                        "project_connection_id": "<CONNECTION>"
                    }
                ]
            }
        },
        {
            "type": "mcp",
            "server_label": "github",
            "server_url": "https://api.githubcopilot.com/mcp",
            "project_connection_id": "<CONNECTION>"
        }
    ],
)
print(toolbox_version.mcp_endpoint)
```

7.6 Consume from any framework

```
mcp_tool = MCPStreamableHTTPTool(
    name="toolbox",
```

```

url=TOOLBOX_ENDPOINT,
http_client=httpx.AsyncClient(
    auth=_ToolboxAuth(token_provider), # injects Entra bearer per request
    headers={"Foundry-Features": "Toolboxes=V1Preview"},
    timeout=120.0,
),
load_prompts=False,
)
agent = chat_client.as_agent(
    name=AGENT_NAME,
    instructions=SYSTEM_PROMPT,
    tools=[mcp_tool],
)

```

The agent **discovers all bundled tools dynamically** at runtime — no per-tool wiring, no custom SDK per integration. Authentication options include **OAuth identity passthrough** and **Microsoft Entra Agent Identity**.

□ **Foundry-homed, not Foundry-bound.** Toolboxes are configured and governed in Foundry, but consumed by any MCP-capable runtime — Microsoft Agent Framework, LangGraph, custom Python, **GitHub Copilot**, **Claude Code**, **MCP-enabled IDEs**, and so on.

7.7 Promote to default

```

client.beta.toolboxes.update(
    toolbox_name="customer-feedback-triaging-toolbox",
    default_version="<version_id>",
)

```

7.8 Tool governance — fillable spec template

Most agent failures come from **under-specified tools**, not the model. Before exposing a tool through Toolbox, fill the spec below. Keep it in your repo next to the tool definition; review it in PRs.

Field	Example	Why it matters
Tool name	crm.create_ticket	Stable identifier; agents and audits reference it
Purpose (1 sentence)	Open a Tier-2 support ticket against the production CRM	Forces clarity; rejects vague tools
Input schema	JSON-Schema with required + optional fields, types, regex patterns	Validated by Toolbox before model can hallucinate parameters
Output schema	JSON-Schema; document error envelope shape	Lets the agent reason about results / failures consistently

Field	Example	Why it matters
Authentication	OAuth on-behalf-of user / Entra Agent ID / API key in KV	No tool ever uses a long-lived secret in code
Authorization scope	Allow-list of agents (or Toolbox group) permitted to call	Default-deny; smallest scope that works
Timeout	8 s soft / 15 s hard	Bounds latency contribution to the agent turn
Retry policy	2 retries, exponential backoff with jitter; only on idempotent ops	Prevents thundering-herd and double-execution
Idempotency	Yes / No; if yes, declare the idempotency key field	Critical for any side-effect tool
Side effects	None / Reads only / Writes / External notification	Drives the human-approval requirement
Risk level	Low / Medium / High / Critical	Drives review depth + monitoring
Human approval required?	No (Low/Med) / Yes (High) / Mandatory + ticket (Critical)	Mark <code>requires_approval: true</code> in <code>agent.yaml</code>
Logs emitted	Tool name · agent version · principal · params hash · status · latency	Mandatory for audit + KQL alerting
PII exposure	None / Pseudonymous / Direct PII / Sensitive personal data	Drives Purview labels + DLP rules
Failure behavior	Return structured error · circuit-break after N consecutive failures · degraded mode	Agent must surface a useful message, not a stack trace
Owner team / on-call	@team-crm-platform, on-call rotation link	Incident routing

Operational rule: a tool without **all 15 fields filled** is not allowed in any production toolbox.

8 Hosted Agents in Foundry Agent Service

What you will learn

- How Hosted Agents differ from a traditional container deployment (per-session sandbox, scale-to-zero, persistent FS, per-agent Entra ID).
- The Sessions vs Conversations distinction and how to pick a protocol (Responses vs Invocations vs A2A).
- The versioning + canary primitives the platform exposes — and what is still Public Preview.

Status: Public Preview (Foundry Agent Service control plane is GA; the hosted-agents compute / runtime described here remains in public preview as of June 2026 — see

the [official Hosted Agents \(preview\) page](#)) · **Production readiness:** Suitable for pilots and pre-production hardening. **Do not commit external production SLAs to hosted-agents until GA.**

This is the runtime intended for production once Hosted Agents reach GA. Today (June 2026) it is suitable for pilots and pre-production hardening. The 2026 refresh is materially different from the original Ignite preview — it now provides **per-session VM-isolated sandboxes, persistent filesystem, scale-to-zero with state resume**, and **dedicated Entra Agent IDs** out of the box.

8.1 Why traditional compute doesn't fit agents

Containers, App Service, serverless functions were designed for **shared multi-tenant web traffic**. Agents have a fundamentally different access pattern:

Concern	Traditional compute	Hosted Agents
Isolation	Multiple sessions share a container	Every session gets a dedicated VM-isolated sandbox
Cold start	Seconds to minutes (high variance)	Seconds (low variance, predictable)
Idle cost	Always-on billing or slow scale-from-zero	Scale-to-zero with FS-preserving resume
State persistence	You build it (DBs, blob, ...)	Built in — files & disk survive scale-to-zero
Identity	Shared service account	Per-agent Entra ID + per-user (OBO)
Observability	You build it	Built in (agent / session / fleet)

8.2 Core capabilities (2026 refresh)

- **Predictable cold starts** — fast warm-up in your custom Docker image. **Measure in your target region and container size before committing latency SLOs** — Microsoft does not currently publish a guaranteed cold-start ceiling.
- **Scale to zero — no Hosted Agents CPU/RAM charge while idle**; agents suspend between turns. Persistent storage, ACR, monitoring (Log Analytics / Application Insights), tools, memory and model usage continue to be billed separately.
- **Resume with filesystem intact** — \$HOME and uploaded /files survive scale-to-zero events
- **Per-session isolation** — every session gets a dedicated, hypervisor-isolated sandbox; isolation keys to namespace end-user sessions
- **BYO VNet** — pin outbound traffic through your own virtual network for private connectivity
- **Production endpoints** — built-in version management with **stable endpoints** + **weighted rollout** between versions
- **Multiple protocols** — Responses (OpenAI-compatible) + Activity (Teams/M365) + Invocations (custom JSON / AG-UI / webhooks) + A2A — all combinable in one agent

8.3 Container model

You package your agent as a **container image**, push it to **Azure Container Registry**, and reference it from `agent.yaml`. The platform pulls the image, provisions compute on demand, assigns a per-

agent **Entra Agent ID**, and exposes a dedicated endpoint:

Responses : {project_endpoint}/agents/{name}/endpoint/protocols/openai/v1/responses

Invocations : {project_endpoint}/agents/{name}/endpoint/protocols/invocations

Active endpoints depend on the protocols declared in the version definition (agent.yaml when using azd, or container_protocol_versions via the SDK).

8.4 Identities involved

Identity	Scope	Purpose
Agent Entra identity (per agent)	Created automatically at deploy	Runtime auth — model invocation, tool access, downstream Azure
Project managed identity (per project)	System-assigned on the Foundry project	Platform infrastructure ops (e.g., ACR Repository Reader)

When you deploy with azd, the **Foundry User** role (formerly *Azure AI User* — the Foundry RBAC roles were renamed; role IDs and permissions are unchanged) is assigned automatically to the agent's Entra identity at account scope. For **external resources** (your own Storage, Cosmos, Key Vault), you assign RBAC manually to that same agent identity.

8.5 Sessions vs Conversations

This is a critical distinction. They serve different concerns and behave differently per protocol.

Concept	What it is	Where it lives	Lifetime
Session	Logical session with persisted compute state — \$HOME, uploaded files	Tied to compute (provisioned/deprovisioned automatically)	Idle timeout 15 min ; long wall-clock supported (verify ceiling on the limits page)
Conversation	Durable record of messages, tool calls, responses	Stored in Foundry; independent of compute	As long as you keep it

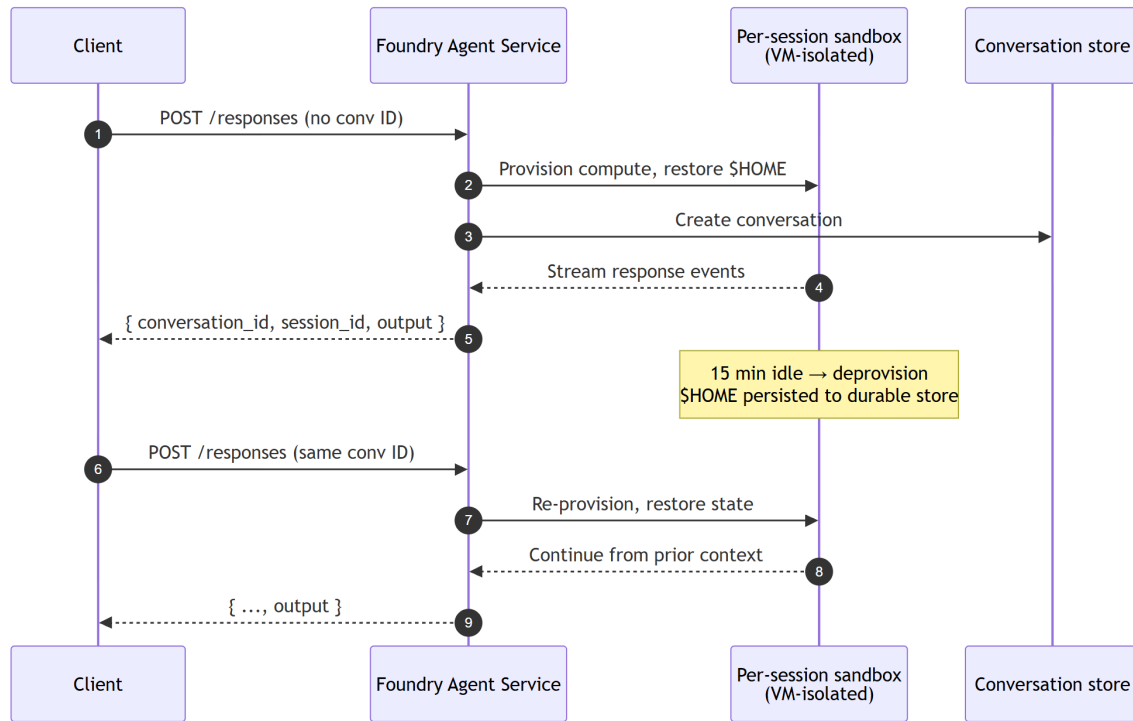


Figure 4: Diagram 4

Responses protocol — conversation ID is primary; the platform manages history; session ID is returned for /files uploads. **Invocations protocol** — session ID is primary; you manage state; no platform conversation history.

8.6 Choosing a protocol

Scenario	Protocol	Reason
Conversational chatbot or assistant	Responses	Platform-managed history, streaming, lifecycle
Multi-turn Q&A with RAG or tools	Responses	Built-in conversation threading + tool result handling
Background / async processing	Responses	background: true with platform-managed polling
Agent published to Teams or M365	Responses + Activity	Responses for logic; Activity for the channel
Webhook receiver (GitHub, Stripe, Jira)	Invocations	Caller's payload format, can't be reshaped
Classification / extraction / batch	Invocations	Structured-data in / structured-data out
Custom streaming protocol (AG-UI)	Invocations	Raw SSE control needed
Protocol bridge / proprietary clients	Invocations	Caller defines the contract

□ Not sure? Start with **Responses**. You can always add an Invocations endpoint later in the same agent.

8.7 Versioning, canary, and blue/green

Every create version produces an **immutable agent version** — a snapshot of the image, resources, env vars, and protocol config. Deployments reference a specific version. To update, you create a new version; to roll out gradually, you split traffic.

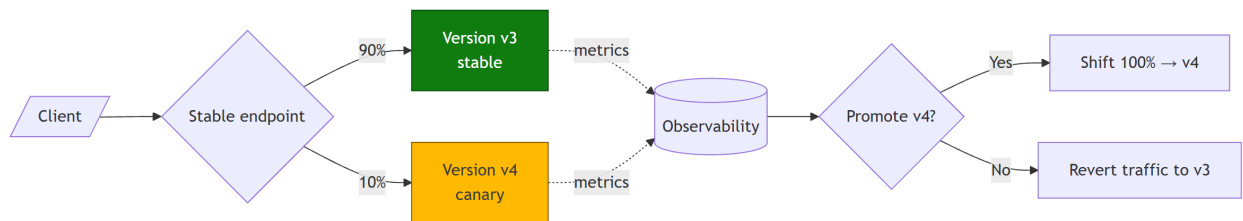


Figure 5: Diagram 5

8.8 One-command deployment

```
azd deploy
```

That single command: - Builds the container image and pushes to ACR - Creates a new immutable agent version - Provisions the per-agent Entra identity - Assigns the Foundry User role (formerly Azure AI User) - Wires up the stable endpoint - Applies the deployment configuration supported by the current `azure.ai.agents` extension

Note: CI/CD promotion gates, evaluation gates and rollback policies are **not** automatically configured by `azd deploy`. Implement them explicitly in your pipeline (GitHub Actions / Azure DevOps) — see the promotion workflow further down.

9 End-to-End Reference Architecture

What you will learn

- The reference architecture across channels, control plane, data plane, tools, enterprise context, security and observability.
- Layer responsibilities at a glance, including which boxes are GA and which are Public Preview.

The diagram below — built with Azure & Foundry icons in draw.io — captures the full reference architecture: channels, control plane (Foundry project, Toolbox, Memory), the hosted-agent runtime in a BYO VNet, governed data and tools, and the observability backbone. The editable source is at `drawio/Foundry_Agent_Reference_Architecture.drawio`.

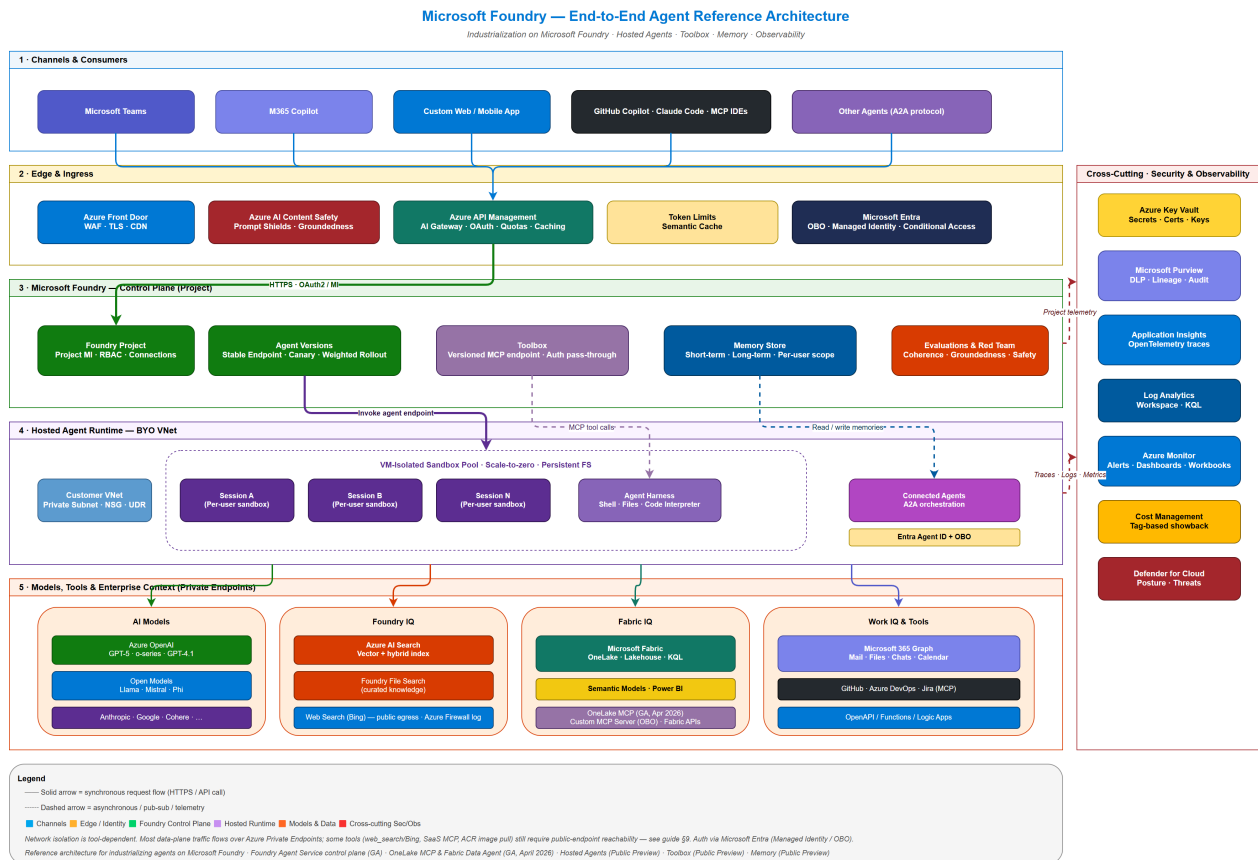


Figure 6: End-to-end Foundry Agent reference architecture

The mermaid view below captures the same flow as a quick-reference for in-document reading.

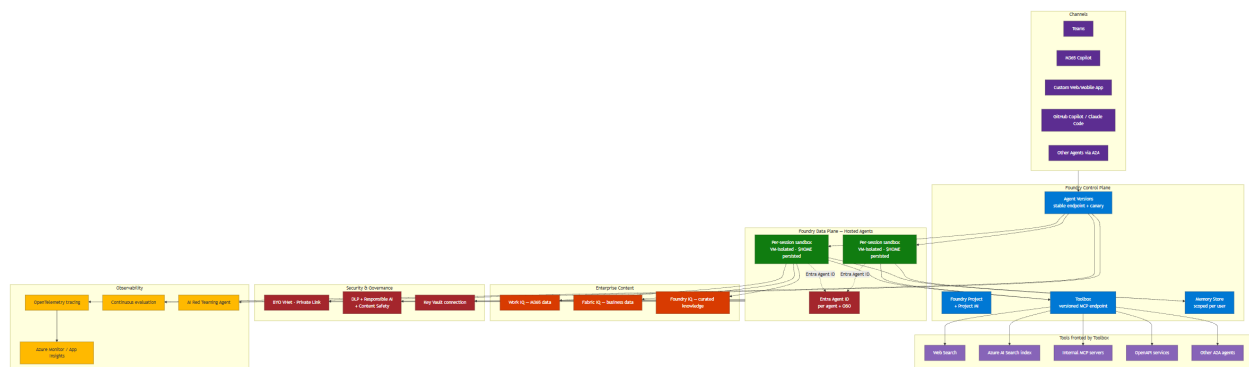


Figure 7: Diagram 6

9.1 Layer responsibilities

Layer	What it owns
Channels	User-facing entry points — published once, consumed many ways
Foundry Control Plane	Project, agent registry/versions, Toolbox catalog, Memory store config
Hosted Agents (data plane)	Per-session VM-isolated sandboxes with \$HOME persistence + per-agent Entra ID
Toolbox	Single MCP endpoint exposing curated, governed, multi-protocol tools
Enterprise Context	Work IQ (M365), Fabric IQ (business data), Foundry IQ (curated knowledge & search)
Security & Governance	BYO VNet, DLP, Responsible AI, Content Safety, Key Vault
Observability	OpenTelemetry, evaluation, red-teaming, Azure Monitor / App Insights

10 Industrialization with azd and CI/CD

What you will learn

- A recommended (not enforced) project layout for Hosted Agents.
- The three tiers of the azd command surface — what is GA, what is preview, what you must implement yourself.
- A CI/CD promotion workflow with explicit eval, security and approval gates.

10.1 Recommended project structure

This guide does **not** ship with a starter repository. The layout below is a recommended convention — adapt it to your existing engineering standards. There is no official Microsoft template enforced for Foundry agents.

```

my-agent/
├── src/
│   ├── agent.py           # MAF / LangGraph code
│   ├── responses_handler.py # Responses protocol endpoint
│   └── tools/              # local tool wrappers
├── Dockerfile              # custom runtime + dependencies
├── agent.yaml               # protocols, resources, env vars, version
├── azure.yaml               # azd service mapping
├── infra/
│   ├── main.bicep          # Foundry project, ACR, VNet, RBAC
│   └── modules/

```

10.2 Key pieces of agent.yaml

YAML below is illustrative. The exact field names, structure and supported keys depend on the installed `azure.ai.agents` `azd` extension version. **Validate against the schema published with the extension version you have installed before copy-pasting into a real pipeline** — preview schemas evolve.

```
name: customer-success-agent
container:
  image: ${AGENT_IMAGE}
  protocols:
    - responses
    - activity
  resources:
    cpu: 1
    memory: 2Gi
env:
  - name: FOUNDRY_PROJECT_ENDPOINT
    value: ${PROJECT_ENDPOINT}
  - name: FOUNDRY_TOOLBOX_ENDPOINT
    value: ${TOOLBOX_ENDPOINT}
  - name: AZURE_OPENAI_DEPLOYMENT
    value: gpt-4o
networking:
  vnet:
    subnet_id: ${AGENT_SUBNET_ID}
versioning:
  rollout:
    strategy: weighted
    canary_percent: 10
```

10.3 Promotion workflow (GitHub Actions)

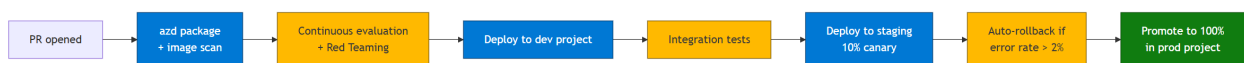


Figure 8: Diagram 7

10.4 One-command developer flow

The `azd` command surface for Foundry agents is split into **three tiers** — be explicit about which one you are using:

Tier 1 — Documented, safe to script in CI/CD:

```
# Bootstrap a fresh agent project from a template
azd init --template foundry-hosted-agent-quickstart
```

```
# Provision infrastructure + deploy in one go
```

```

azd up

# Iterate: rebuild image + push + create new agent version
git commit -am "improve triage prompt"
azd deploy

# Tear it all down
azd down

```

Tier 2 — Foundry agent extension commands (preview surface, verify version):

```

azd ai agent show          # describe deployed agent
azd ai agent invoke --message "ping"
azd ai agent run --evaluate ./evals/triage.yaml
azd ai agent monitor       # tail traces / metrics

```

□ The `azd ai agent ...` extension is in preview — pin a known version of the extension in CI and re-validate after each upgrade.

Tier 3 — Recommended deployment patterns (no dedicated CLI; do them via config + `azd deploy`):

Need	How to do it today
Canary rollout	Set <code>deployment.strategy: weighted</code> in <code>agent.yaml</code> (see §27); <code>azd deploy</code> applies the new weights
Promote canary to 100%	Edit weights in <code>agent.yaml</code> ; commit; <code>azd deploy</code>
Rollback	Re-deploy the previous <code>agent.yaml</code> revision via <code>azd deploy</code> (or revert the commit and re-run the pipeline)
Per-environment deploy	Use <code>azd env new <env></code> per dev/test/prod; same template, distinct subscriptions / resource groups

11 Security & Governance

What you will learn

- The trust and identity boundaries an agent crosses (tenant → project → sandbox → Toolbox → SaaS) and which identity actually carries each hop.
- What is private and what stays public in a BYO VNet deployment, including the Bing egress reality.
- Defenses against prompt injection, indirect injection, exfiltration, hallucination, excessive agency, and token abuse.

Treat a hosted agent as **production application code**.

11.1 Trust & identity boundaries — at a glance

Before the per-layer details below, the diagram shows which identity actually crosses each boundary — tenant → Foundry project → per-session sandbox → Toolbox → SaaS. The same identity rarely survives the full hop; that is by design.

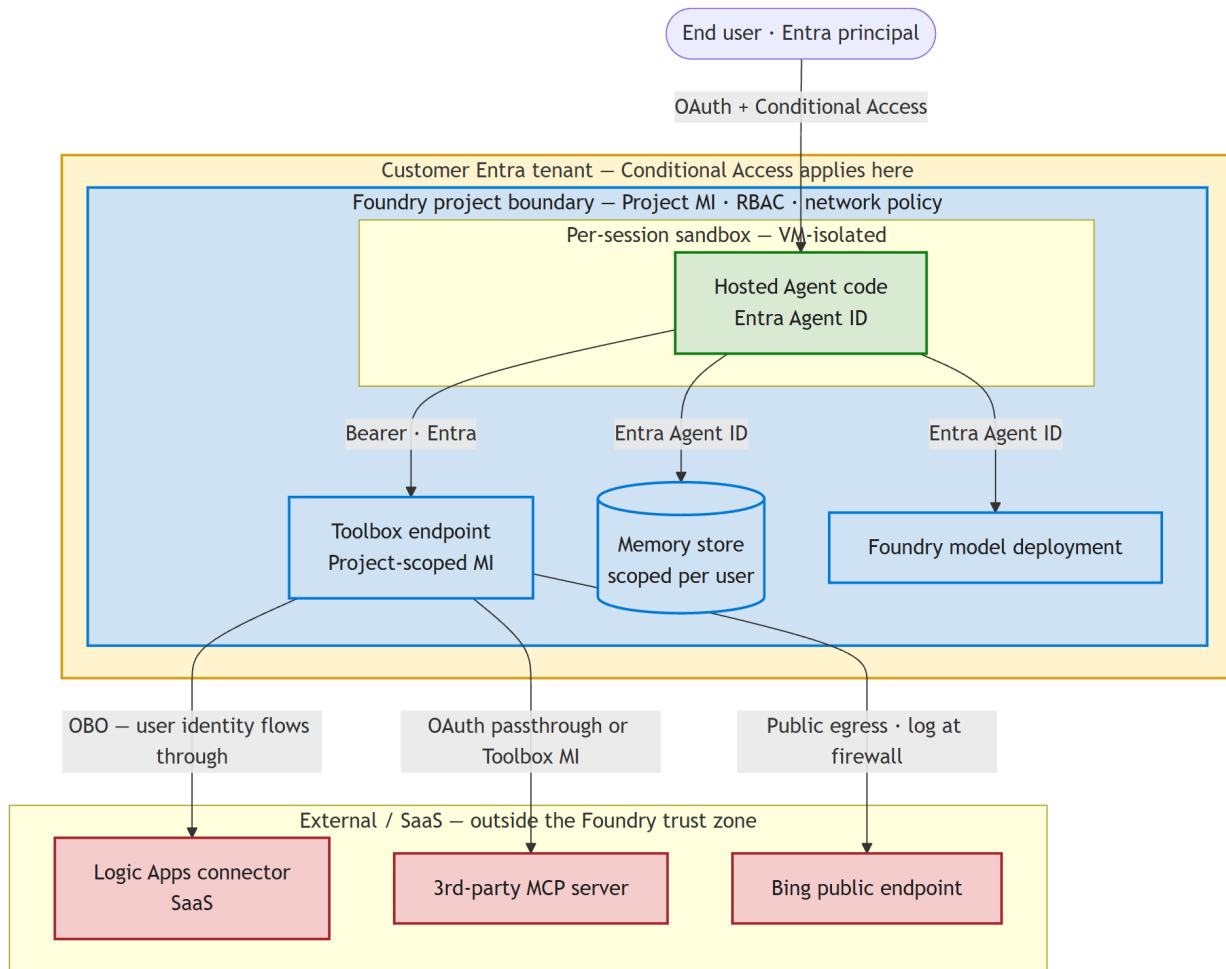


Figure 9: Diagram 8

11.2 Identity & access (no shared service accounts)

Layer	Mechanism
Agent → Foundry models	Per-agent Entra Agent ID (auto-assigned at deploy)
Agent → user data	OBO (On-Behalf-Of) flow — auditable, revocable per user

Layer	Mechanism
Agent → Azure resources (Storage, KV, Cosmos...)	Manual RBAC to the agent's Entra identity
Agent → Toolbox	Bearer token via Entra; toolbox forwards via OAuth passthrough or its own MI
Tool → external SaaS	OAuth identity passthrough configured once in Toolbox

11.3 Network isolation — what's in, what's still public

- **BYO VNet** — for the hosted agent runtime, **outbound agent traffic can flow through your VNet** (preview), enabling Private Endpoints to your data sources
- **Private Link** is supported toward Foundry, ACR, Key Vault, Storage and Cosmos (when those resources are configured with PE)
- **Caveats — not everything is private today:**
 - **Foundry-managed file_search** BYO VNet support is **evolving** — **verify on the Foundry network isolation page before designing around it**. If the capability you need is not there, fall back to **Azure AI Search with Private Endpoint** exposed through the Toolbox
 - **web_search / bing_grounding** intentionally calls a **public endpoint** (Bing) — there is no "private web". Allow-list domains, force outbound traffic through your Azure Firewall (or NVA), and **monitor the Bing FQDN at the firewall** so every query is logged and inspectable. Combine with Content Safety + URL allow/block lists in Foundry
 - **Logic Apps connectors and many SaaS MCP servers** are public SaaS endpoints by design — apply DLP and JIT tokens, not network isolation
 - **Code Interpreter and File Search via Toolbox** in hosted agents have specific multi-tenant isolation considerations — review the official docs before shipping a multi-tenant SaaS

11.4 Content safety & DLP — what is built in vs what you must configure

- **Built-in (project-level)** — **Azure AI Content Safety** with **Prompt Shields** (direct + indirect prompt injection), jailbreak detection, harmful-content categories, groundedness detection, **Responsible AI** abuse monitoring
- **Configurable** — content-filter strictness per category, custom blocklists, region selection, allowed/blocked domains for web_search
- **Not automatic** — **Microsoft Purview / DLP** integration is a *connect-and-configure* exercise, not a default. Plan an explicit project task to wire Purview labels, DLP policies and audit log routing to your central SIEM
- **Defense in depth** — never rely on a single layer. Combine prompt-side (Prompt Shields), tool-side (allow-listed and approval-gated tools), data-side (RBAC + Purview labels) and network-side (Private Endpoint + DLP) controls

11.5 Secrets

- Never put secrets in container images or env vars. □ Use **Foundry Key Vault connections** + Managed Identity for any secret-bearing dependency.

11.6 Third-party tool risk

If your agent calls non-Microsoft tools or models, **review what data flows out** — retention, location, governance. Microsoft makes no representation about third-party data handling. Apply DLP and Content Safety at the agent boundary.

11.7 Defending against LLM-specific attacks

Prompt-based threats need prompt-aware defenses — RBAC and network isolation alone are not enough.

Attack class	Defense in Foundry
Direct prompt injection	Azure AI Content Safety – Prompt Shields (jailbreak detection) enabled at the agent level — blocks user attempts to override system instructions
Indirect prompt injection (poisoned doc / web page / tool output)	Prompt Shields document-protection mode scans tool responses and grounded content for injected instructions before they reach the model
Data exfiltration via tool calls	Toolbox enforces allow-listed tools per agent . DLP must be designed and tested explicitly — Foundry does not automatically intercept every tool payload. Combine: per-tool egress allowlists, Foundry Tools outbound URL controls, APIM policies on the front door, Purview DLP/labels where applicable, PII redaction. See the network and DLP framing earlier in this section
Hallucination / ungrounded output	Groundedness evaluator in CI/CD (§25); Content Safety groundedness detection at runtime can flag or block low-grounded responses
Excessive agency / unintended actions	Mark high-impact tools as requires_approval: true in <code>agent.yaml</code> → forces human-in-the-loop confirmation through Foundry Approvals
Token abuse / cost-of-service	Per-project token budget alerts (§13); per-agent rate limiting at API Management front-door

11.8 Per-agent least privilege — concrete pattern

Treat the **Entra Agent ID** as you would an Azure VM identity:

1. One agent = one Entra Agent ID (auto-issued at deploy)
2. Grant **only the Foundry connections** the agent must reach (named connections, not wild-cards)

3. For each Azure resource (Storage, Cosmos, Key Vault, AI Search): Azure RBAC role with the **smallest applicable scope** (resource > resource group > subscription) and the **smallest applicable role** (Storage Blob Data Reader ≠ Contributor)
4. **No long-lived secrets in agent.yaml** — declare a `keyvault_connection:` and reference secret names; agent fetches at boot via MI
5. Rotate model API keys via Key Vault references; agents never see the raw key

11.9 Audit trail per tool call

OpenTelemetry traces (§12) capture **every tool invocation** with: agent version, Entra principal, tool name, input/output (subject to DLP redaction), latency, status. Sink to Log Analytics with **immutable retention** (Azure Storage immutability policy or Microsoft Sentinel archive) for forensic-grade audit. KQL query example:

```
AppTraces
| where Properties['ai.operation.name'] startswith "tool."
| project TimeGenerated, AgentVersion=Properties.agent_version,
        Tool=Properties.tool_name, Caller=Properties.user_oid,
        Status=Properties.status, LatencyMs=DurationMs
| where Status != "success"
| summarize Failures=count() by AgentVersion, Tool, bin(TimeGenerated, 5m)
```

12 Observability & Operations

What you will learn

- The OpenTelemetry data flow from agent to App Insights / Log Analytics / Foundry tracing UI.
- The five baseline alerts every production agent should emit, and where the gaps are versus a classical web service.

Observability is built around **OpenTelemetry**. Core tracing and continuous evaluation are GA; the **AI Red Teaming Agent** and some advanced evaluators remain in Public Preview — verify per-feature status on Microsoft Learn before relying on them for compliance or contractual evidence.

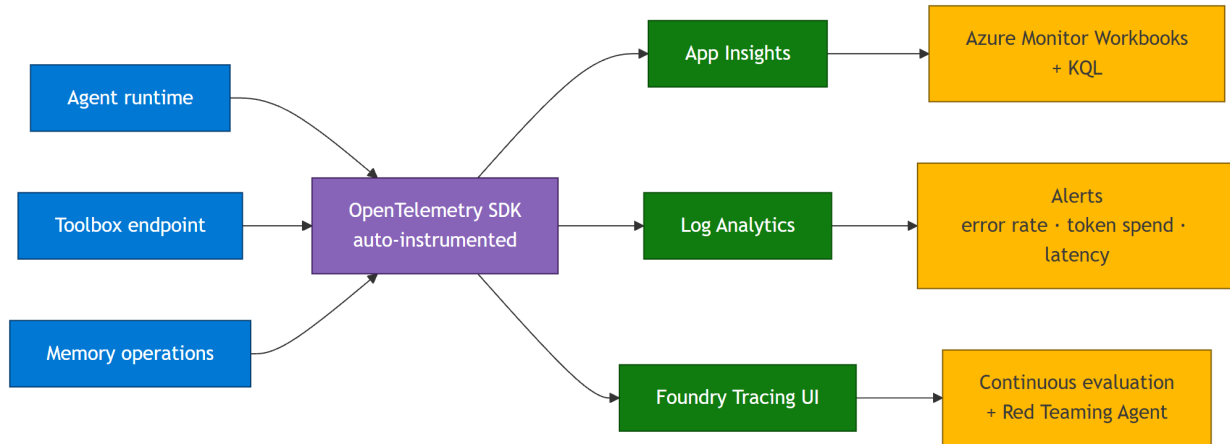


Figure 10: Diagram 9

12.1 What's in scope

Signal	Captured by	Use
End-to-end traces	OpenTelemetry auto-instrumentation across agent, toolbox, model calls	Debug latency, follow a request through a multi-agent workflow
Per-tool invocation traces	Toolbox MCP layer	Spot tool errors, slowness, governance violations
Token & cost metrics	Foundry tracing	Per-agent, per-conversation cost attribution
Continuous evaluation	Foundry Control Plane	Quality regressions caught before promotion
Adversarial coverage	AI Red Teaming Agent	Automated attack scenarios on each version
Alerts	Azure Monitor	Error spikes, token-budget exhaustion, model unavailability

12.2 Recommended baseline alerts

1. Agent version error rate > 2% over 5 min
2. Tool call P95 latency > 10 s sustained
3. Token spend per project > 80% of monthly budget
4. Memory retrieval failure rate > 1%
5. Red-teaming regression on any new version (block promotion)

13 Cost & FinOps

What you will learn

- Why Hosted Agent compute is < 2% of TCO and the model deployment dominates the bill.
- The cost levers (scale-to-zero, sandbox sizing, Toolbox reuse, model selection, Model Router, batching & caching, eval gating, PTU vs PAYG) and the Build 2026 Select→Evaluate→Optimize→Operate→Improve loop.

Hosted Agents cost economics are different from traditional compute, but **model spend dominates the bill** — the worked example in §21 shows sandbox CPU + RAM at < 2 % of TCO. Optimise model deployment mode and selection first; tune sandbox sizing second.

Build 2026 framing — cost is a first-class architectural concern. Treat model selection and optimization as a continuous loop — **Select** → **Evaluate** → **Optimize** → **Operate** → **Improve** — not a one-off. The system-level levers Microsoft calls out are: **intelligent routing** (Model Router), **batching** (async for non-real-time work), **caching** (don't pay twice for identical requests), **provisioned throughput** for predictable capacity, **quota management** (quota tiering, global / data-zone customer quota), and **model optimization** (compression, fine-tuning, distillation). Profile cost *by task type* before optimizing globally — routing is workload-specific.

Cost lever	Mechanism	Impact
Scale-to-zero	Compute deprovisioned after 15 min idle; you pay \$0	Removes always-on baseline cost
Per-session sandbox	You pay only for active execution per session	Linear cost per real user activity
Per-agent right-sizing	cpu / memory declared per version	Avoid over-provisioning; tune by workload
Toolbox reuse	One implementation per tool, shared across agents	Removes duplicated dev cost & duplicated egress
Memory-as-a-service	Foundry-managed; you pay \$0.25/1K events + retrievals	No DB to provision, scale, secure
Model selection per agent	Multi-model — pick the cheapest model that meets quality	Cost per 1K tokens varies 10× across models
Model Router	Route each request to the right model by complexity/budget	Sends only hard requests to expensive models
Batching & caching	Async batch for non-real-time work; cache identical/near-identical requests	Cuts repeated and peak-time spend
Continuous eval gating	Block costly low-quality versions before promotion	Quality-cost optimization in CI/CD

13.1 Memory pricing reminder (effective June 1, 2026)

Component	Price
Short-term memory	\$0.25 per 1K events stored

Component	Price
Long-term memory	\$0.25 per 1K memories per month
Memory retrieval	\$0.50 per 1K retrievals

13.2 Patterns to adopt early

- **Tag agents and conversations** with cost-center metadata for chargeback
- Set **per-project token budgets** in Azure Monitor with alerts at 80% / 100%
- Use **canary rollouts** to detect cost regressions before they hit 100% of traffic
- **Compact context** aggressively for long sessions — token cost grows quadratically without it

14 Decision Matrix — Hosted Agent vs Prompt-Based Agent

What you will learn

- When **not** to build an agent at all (and the cheaper primitive to use instead).
- How to choose between Prompt-based, Workflow, and Hosted Agent on cost, control, and ops fit.

14.1 When *not* to build an agent at all

The first decision is whether you actually need an agent. **Agents add latency, cost and surface area** — pick a simpler primitive when you can.

Symptom	Better than an agent	Why
Stateless transformation, deterministic input/output	A plain Azure Function or REST API	No LLM-at-runtime cost, deterministic SLO, easier to test
Multi-step process with fixed branches	A Logic App / Durable Function workflow	Deterministic, retries built in, much lower cost
Pure RAG ("answer from these docs")	A classic RAG endpoint (AI Search + a model call)	An "agent" wrapper adds cost without changing the answer
Scheduled / batch job	A Function timer + queue	Hosted agents are interactive sandboxes; not ideal for cron
Process that must be fully auditable per step	A workflow agent (declarative steps), not a free-roaming hosted agent	Each step is observable and policy-checked
Use case requiring legally-binding decisions	A human-in-the-loop tool with the LLM as advisor	Liability and compliance constraints

Choose an agent when you need autonomous tool selection across an open task space, multi-turn dialogue with state, dynamic tool composition, or natural-language UX.

14.2 Hosted Agent vs Prompt-Based Agent

You need to...	Choose	Why
Define an agent in prompts + tool config only	Prompt-based agent	No-code, stays in the portal
Bring your own framework (MAF, LangGraph, SK, custom)	Hosted Agent	Containerized — bring any code
Accept webhooks or non-OpenAI payloads	Hosted Agent + Invocations	Custom payload contract
Specify CPU/memory for the sandbox	Hosted Agent	Resource control per version
Persist files/state across turns (\$HOME, /files)	Hosted Agent	Per-session FS persistence
Run inside your own VNet	Hosted Agent	BYO VNet supported
Need per-agent Entra ID + per-user OBO with custom code/runtime	Hosted Agent	Hosted agents get a dedicated Entra Agent ID and let you implement OBO in code. Note: Agent Applications (published prompt or workflow agents) also receive a unique Entra agent identity at publish time — re-evaluate before assuming Hosted is the only option for identity alone

15 Test & Simulation

What you will learn

- How to inject deterministic failures (429, tool 503, tool malformed payload) before promotion.
- How to wire Red Teaming + Chaos Studio into the gating pipeline.

Before promoting a new version, exercise the rough paths.

15.1 Inject failures at the protocol layer

```
# Force a model 429 in a test environment
import httpx, pytest
from agent_framework.testing import MockResponsesClient

mock_client = MockResponsesClient(
    responses=[httpx.Response(429, json={"error": "rate_limit_exceeded"})]
)
agent = mock_client.as_agent(name="trriage", instructions="...")
```

```
with pytest.raises(RetryableError):
    await agent.run("test prompt")
```

15.2 Validate canary metrics with KQL

```
AppInsights_Traces
| where customDimensions.agent_name == "customer-success-agent"
| where customDimensions.agent_version in ("v3", "v4")
| summarize
    error_rate = countif(customDimensions.outcome == "error") * 100.0 /
    count(),
    p95_latency = percentile(durationMs, 95)
by tostring(customDimensions.agent_version)
```

15.3 Adversarial coverage

Wire the **AI Red Teaming Agent** into your promotion gate. Each candidate version is exercised with attack scenarios; promotion is blocked on regression. Combine with continuous evaluation suites for quality regressions.

15.4 Chaos for downstream services

Use **Azure Chaos Studio** to inject faults in dependent services (model regional outage, Toolbox tool 503, memory store throttling) to validate that the agent degrades gracefully — surfaces a useful error to the user, retries with backoff, and emits actionable telemetry.

16 Anti-Patterns to Avoid

What you will learn

- The recurring failure modes seen across architecture, tooling, data, and governance.
- The PR checklist to apply at code-review time so they don't reach production.

A guide that only shows the happy path is dangerous. The following are the **recurring mistakes** that turn promising agent pilots into production incidents — recurring failure modes seen in enterprise agent programs.

16.1 Architecture anti-patterns

Anti-pattern	Why it fails	Do this instead
Wrap a simple API in an agent without a clear agent-shaped reason One mega-agent handling everything	LLM cost + latency + non-determinism for zero benefit Tool sprawl, exploding context window, impossible to test or audit	Plain Function or REST endpoint (see §14 “When NOT to build an agent”) Split into focused agents with A2A composition
Use a hosted agent for a deterministic multi-step process Skip Toolbox and call tools directly from each agent Hardcode the model name in agent code (e.g. literal "gpt-4o")	Pay sandbox + LLM for what a Logic App does cheaper and reliably 5 agents × 5 tools = 25 integration points to maintain Cannot swap model at deprecation or for cost optimization without a code release	Workflow agent or pure Logic App One Toolbox per domain, consumed by all agents Read from env var (AZURE_OPENAI_DEPLOYMENT) and use a stable deployment alias (prod-chat-model); model swap = config change, not code change
Assume Foundry agent and consumed services live across tenants	Cross-tenant Entra trust is rarely supported during preview; broken auth, broken OBO	Foundry project, Hosted Agent, Toolbox, Memory, MCP servers and connected data sources should be in the same Entra tenant unless a feature explicitly documents cross-tenant support

16.2 Tooling anti-patterns

Anti-pattern	Why it fails	Do this instead
Connect any MCP server you find on the web	Supply-chain risk: untrusted code running with your agent's identity	Allow-list MCP servers; review code; pin versions; SBOM scan
Give an agent 50 tools “just in case”	Model confusion, tool-selection errors, blown context window	≤ 15 tools per agent; specialize agents instead
Tools without idempotency keys for write actions	Retries duplicate orders, send double emails, double-charge cards	Mandatory idempotency key for any side-effect tool
No human approval on irreversible actions	Agent deletes data / sends external comms / pays a vendor	Mark <code>requires_approval</code> : true for any High/Critical-risk tool

16.3 Data & memory anti-patterns

Anti-pattern	Why it fails	Do this instead
Memory with no scope	One user's preferences leak into another's session	Always scope by user / tenant / workspace; contract-test the boundary
Memory with no retention policy	Stale facts persist forever; "right to be forgotten" is impossible	TTL per memory class + scheduled cleanup + GDPR-ready CRUD
Trust the agent's "memory" over the system-of-record	Agent confidently states an outdated price / status	Memory is never authoritative ; cross-check sources
Send raw secrets in the prompt or env vars	Secrets end up in traces, logs, model providers	Always Key Vault + MI; reference by name

16.4 Process & governance anti-patterns

Anti-pattern	Why it fails	Do this instead
Ship a preview-status capability into a critical workload without a GA cutover plan	No SLA when it breaks; pricing surprises at GA	Pin GA milestones; document fallback; set budget alerts
"Eval = a few manual prompts in the playground"	Quality regressions ship undetected; cost blows up	Curated golden dataset + automated evaluators in CI
Confuse technical observability with business quality	Latency is fine but answers are useless	Track both: traces <i>and</i> per-conversation quality scores
No incident runbook for the agent	First incident = blind panic; rollback takes hours	Pre-written runbook covering rollback, model swap, key revoke
Treat the model as immutable	Model deprecates with 6-month notice; you scramble	Quarterly model-sweep eval; alias-based deployment names
Multi-tenant SaaS without isolation testing of every tool	Tenant A sees Tenant B's data through code_interpreter or file_search	Per-tool isolation review; contract test every cross-tenant boundary

Use this list as a PR checklist. If a change introduces any of the above, require justification + sign-off before merge.

17 Production Readiness Checklist

What you will learn

- The pre-flight items (architecture, identity, tools, state, ops, distribution) that should all be green before traffic.

- The verify-before-go-live sub-checklist — concrete actions on top of the checkbox list.

17.1 Architecture

- ☐ Hosted Agent packaged as a versioned container image in **ACR**
- ☐ Two protocols at most per agent (avoid sprawl); justify each
- ☐ Stable endpoint exposed; canary slot configured
- ☐ BYO VNet enabled; outbound goes through Private Endpoints

17.2 Identity & Security

- ☐ Per-agent **Entra Agent ID** with least-privilege RBAC
- ☐ User actions flow via **OBO**, not via the agent identity
- ☐ No secrets in image or env vars; **Key Vault connection** in place
- ☐ Content Safety + Prompt Shield enabled
- ☐ DLP and Responsible AI policies applied at the project

17.3 Tools

- ☐ Toolboxes for any tool used by ≥ 2 agents
- ☐ Toolbox versioned; consumers point at the unversioned default endpoint
- ☐ OAuth passthrough or Entra Agent ID auth configured per tool
- ☐ Per-tool tracing visible in App Insights

17.4 State

- ☐ Memory store provisioned with explicit scope per user / tenant
- ☐ CRUD operations exposed for user data-rights requests (read / delete)
- ☐ Context compaction strategy chosen (sliding window or summarization)

17.5 Operations

- ☐ OpenTelemetry traces flowing to App Insights and Foundry tracing UI
- ☐ Continuous evaluation suite gating promotions
- ☐ Red Teaming Agent gating promotions
- ☐ Cost alerts at 80% / 100% of monthly budget per project
- ☐ Runbook for rollback (re-deploy with the previous agent.yaml weights via `azd deploy`)

17.6 Distribution

- ☐ Publication (Teams app · M365 app · external endpoint) gated by signed releases
- ☐ Consumer documentation written; sample requests provided
- ☐ Decommission plan documented (see §27 Decommissioning checklist)

17.7 Verify-before-go-live (must be green before traffic)

- ☐ **Region availability** verified for every model + every Foundry capability used

- ☐ **Quotas** verified against expected peak load (TPM, RPM, files, sessions)
- ☐ **Pricing** re-validated against the live Microsoft pricing page; budget alerts armed
- ☐ **Model fallback** defined (which model the agent switches to if primary is unavailable)
- ☐ **Tool permissions** reviewed line-by-line for least privilege
- ☐ **Human approval** wired for every High / Critical-risk tool
- ☐ **Memory retention policy** documented + cleanup job scheduled
- ☐ **Evaluation dataset** versioned in repo; baseline eval scores recorded
- ☐ **Red-Team Agent** scan passed without new high-severity findings
- ☐ **Logs and traces** reaching central Log Analytics with immutable retention
- ☐ **Incident runbook** written, dry-run executed, on-call rotation in place
- ☐ **Rollback strategy** documented + previous version warm in the weighted-deployment slot
- ☐ **Anti-patterns** review (above) — no item flagged
- ☐ Activity protocol enabled if published to Teams or M365 Copilot
- ☐ A2A endpoint exposed if other agents will delegate to this one

18 Required Personas for Industrialization

What you will learn

- The minimum RACI for an agent platform — who owns what across product, platform, security, and FinOps.

A successful agent platform involves more roles than the agent developer alone. The following RACI shows the minimum set.

Persona	Owns	Typical seat
Foundry Project Owner	Project lifecycle, RBAC at project scope	Platform team
Agent Developer	Agent code, prompts, tools, evaluation suites	Product / app team
Tools Owner	Toolbox curation, tool auth config, governance policies	Platform / integration team
Identity Engineer	Entra Agent ID, OBO flows, conditional access	Security / IAM
Network/Security Engineer	VNet, Private Endpoints, DNS, DLP	Cloud Network team
MLOps / FinOps	CI/CD, cost dashboards, alerts, model selection	Platform / FinOps
Responsible AI advisor	Content Safety policies, Red Teaming review, RAI sign-off	Compliance / Legal / Risk

Persona	Owns	Typical seat
Service / Product Owner	Roadmap, business metrics, channel publication	Business
End-user Support	Triage, escalation, feedback loop into evaluation	Support / Ops

19 Service Limits & Quotas

What you will learn

- The hard documented limits (files, messages, tools, scopes, sessions, idle timeout) and the planning rules-of-thumb on top of them.
- The workflow to raise quotas through the Foundry support path.

The single source of truth is the official [Foundry Agent Service limits, quotas & regions page](#). The tables below separate **documented limits** (hard values from Microsoft docs) from **planning rules-of-thumb** (architectural assumptions you should validate for your workload). Do not treat the second table as a contract.

19.1 Documented limits (verify on the quotas page)

Surface	Documented default	Notes
Files attached per agent / per thread	10,000	Hard ceiling — chunk content or shard threads beyond this Use chunked upload above; offload to Blob/OneLake for very large artefacts
File size per upload (Files API)	512 MB per file	
Total uploaded files per project	300 GB	Aggregate across all agents/threads in the project Past ~100 turns, plan context compaction (\$5)
Messages per thread	100,000	
Tools attached per agent declaration	128 tools	Matches model parallel-tool-call ceiling Hard cap; design scope strategy accordingly
Memory scopes per memory store	100	
Memories per scope	10,000	Run a periodic <code>memory.delete</code> job for stale facts After this, the next call pays cold-start latency
Sandbox idle before scale-to-zero	15 minutes	

Surface	Documented default	Notes
Sandbox session max wall-clock	long-running sessions supported (verify the exact ceiling on the limits page)	Long-running sessions persist \$HOME and /files
Throughput rate-limit responses	HTTP 429 with Retry-After header	Implement exponential backoff with jitter

19.2 Planning rules-of-thumb (architectural assumptions, not contractual)

These are sizing heuristics drawn from preview behavior and field experience. **Confirm against your tenant's actual quota grants and the live quotas page before committing to a production design.**

Dimension	Rule-of-thumb	Notes / mitigation
Concurrent hosted-agent sessions	Soft quota — typically governed per subscription, per region during preview (commonly seen at 50 active concurrent sessions, adjustable via Azure support)	Use isolation keys to namespace per-tenant traffic; track quota at the subscription/region level, not per project
Cold-start sandboxes burst	Predictable but throttled per project	Pre-warm with a synthetic ping during business hours
Sandbox disk per session	~10 GB persistent (subject to change)	Offload large artifacts to Blob / OneLake; keep only working set
Inbound request size (Files API)	~100 MB practical for synchronous uploads	Stream large files from a connected Storage account
Tool calls per turn	Capped by model (e.g. up to 128 parallel calls on GPT-5-class models)	Compose with Toolbox to avoid context bloat
Vector store (Foundry-managed file_search)	preview limit, see quotas page	Use Azure AI Search for larger / VNet-isolated indexes
Toolbox size	30+ tools per Toolbox observed in preview	Split by domain (e.g., crm-toolbox, ops-toolbox) for least privilege
Throughput to model deployment	Bound by PTU / TPM you provisioned on the Foundry deployment	See §21 for PTU vs PAYG sizing

Practical rule: model-side TPM/RPM (provisioned on the Foundry model deployment) is almost always the first wall you hit — not the agent platform. Provision a PTU pool for predictable workloads and front it with APIM (see Foundry_Agent_Monitoring_APIM guide).

19.3 Model quota & shared-quota pools

Azure assigns model quota **per subscription, per region, per model** in **tokens-per-minute (TPM)**; Standard and Provisioned deployments use different mechanics. Manage it in the Foundry portal under **Operate** → **Quota**, which has two tabs — **Token per minute (TPM)** for standard deployments and **Provisioned throughput unit (PTU)** for provisioned ones (with capacity-estimation tooling). After editing an allocation or submitting a request, allow up to **15 minutes** to propagate.

Pool	What it is	Use for
Dedicated quota	Per-subscription/region/model TPM you allocate across projects	Production endpoints
Foundry shared quota	A usage-billed pool you can temporarily draw on, no support ticket	Temporary test endpoints only , not production
Global quota (instant models, preview)	Per-model global pool separate from regional quota; Global Standard deployments reserve a slice, instant models use the remainder	Deployment-less model usage; request a global increase if throttled

Build 2026 also highlights cost/throughput levers that interact with quota: **quota tiering**, **global customer quota** and **data zone customer quota** for more predictable scaling, plus intelligent routing, batching, caching and provisioned throughput. See the [Quota management docs](#).

19.4 Quota-increase workflow

For most model quota, request directly in the portal: **Operate** → **Quota** → **Request quota** (requires **Owner/Contributor** on the subscription; viewing needs **Cognitive Services Usages Reader**). For limits not exposed there:

1. Open an **Azure support request** of type *Service and subscription limits (quotas)*
2. Resource provider: **Microsoft.CognitiveServices** (Foundry) or **Microsoft.MachineLearningServices** (legacy AI Hub projects)
3. Justify with **measured TPM/RPM** from Azure Monitor (not estimates)
4. Allow 24–72 h for approval; quotas raise per-region, per-subscription

20 Regional Availability & SLA

What you will learn

- How to pin region for Hosted Agents + model + Toolbox + data so latency, residency and SLA stay aligned.
- The current SLA surface — control plane GA vs Hosted Agents compute Public Preview.

Hosted Agents and Toolbox are rolling out region-by-region. Pin both your **Foundry project** and your **model deployments** to the same region to avoid cross-region egress, then plan multi-region for HA.

20.1 Region selection checklist

- **Hosted Agents (preview)** — available in a **subset of Foundry regions** that grows frequently (recent additions include East US 2, Sweden Central, Australia East, Canada Central, North Central US, South East Asia, Poland Central, South Africa North, Korea Central, South India, Brazil South, and others). **Do not hardcode a region list in solution design.** Always confirm on the [Foundry region availability page](#) for your selected feature, model, and tool combination at the time of deployment.
- **Models** — not every model is in every region. GPT-5 family and o-series often launch in East US 2 / Sweden Central before others. Cross-check the [Azure OpenAI models matrix](#).
- **Data residency** — choose an EU region (e.g., Sweden Central, France Central, West Europe) when EU Data Boundary is required. EU Data Boundary scope and exceptions are documented separately — read it.
- **Multi-region HA pattern** — deploy two Foundry projects in paired regions (e.g., West Europe + Sweden Central), front with APIM Premium multi-region, and replicate Memory + Toolbox config via IaC. Failover is *active-passive* today; active-active for stateful agents is on the roadmap.

20.2 SLA reference (preview vs GA)

Capability	Status (today)	SLA
Foundry Agent Service control plane	GA	Covered by a Microsoft SLA — verify the current figure in the Azure SLA / Product Terms before quoting any number externally
Hosted Agents compute	Public Preview (June 2026)	No SLA — the Foundry Agent Service control plane is GA, but the hosted-agents compute / runtime is still in public preview. Do not back production SLAs with hosted-agents until GA
Toolbox	Public Preview	No SLA
Memory	Public Preview	No SLA
Microsoft Agent Framework v1.0 (SDK)	GA	N/A (client SDK)

Track GA dates in the [Foundry release notes](#) and lock your production cutover to the GA milestone.

21 Hosted Agent Execution Pricing Detail

What you will learn

- Hosted Agent compute pricing in preview (\$0.0994/vCPU-h, \$0.0118/GiB-h) and what is included.
- A worked TCO example showing why the model deployment, not the sandbox, dominates spend.
- The PTU vs PAYG trade-off when scale-to-zero applies on the agent side.

Section §13 covered the FinOps levers; this section gives the per-unit numbers so you can build a credible cost model.

21.1 Pricing changes frequently — verify before committing

All prices below are **public preview values**, captured from Microsoft announcements at the time of writing. **Always verify the current Hosted Agents pricing on the [Azure AI Foundry pricing page](#) before estimating production costs.** Treat the numbers in this guide as planning assumptions, not as a committed price list.

21.2 Hosted Agent compute (preview pricing — verify on the live page)

These are the public preview prices announced by Microsoft. **They are not GA prices and Microsoft can change them — always re-check the [Azure AI Foundry pricing page](#) before committing a budget.**

Resource	Unit	Public preview price	Comment
Sandbox vCPU	per vCPU-hour active	\$0.0994	Billed only while session is active (scale-to-zero)
Sandbox memory	per GiB-hour active	\$0.0118	Active session only
Sandbox storage (persisted disk)	per GB-month	check pricing page	Even when scaled to zero
Cold-start	no separate charge	—	Warm-up time is billed as normal vCPU-hour. Microsoft does not publish a guaranteed cold-start latency ceiling — measure in your region
Egress via BYO VNet	standard Azure egress	varies	Same as any VNet-bound workload

21.3 Memory pricing — preview, billing scheduled to start June 1, 2026

□ Microsoft Memory is in **public preview**. Pricing structure and effective billing date can change before GA. Treat all numbers below as planning assumptions, not commitments.

Item	Announced preview price
Short-term memory	\$0.25 per 1K events stored
Long-term memory	\$0.25 per 1K memories per month
Memory retrieval	\$0.50 per 1K retrievals

21.4 Worked example — a typical “support triage” agent

Assumptions: 200 sessions/day, avg 3 minutes active, 0.5 vCPU + 1 GiB, 50K tokens/session on GPT-5 PAYG.

Cost line	Daily	Monthly
Compute vCPU (200 × 3 min × 0.5 vCPU × \$0.0994)	~\$0.50	~\$15
Compute memory (200 × 3 min × 1 GiB × \$0.0118)	~\$0.12	~\$3.50
Memory + storage	negligible	~\$5
Model tokens (GPT-5 PAYG, 50K × 200)	~\$60	~\$1,800
Total	~\$61	~\$1,825

The model still dominates. Even with the corrected (higher) compute prices, sandbox CPU + RAM is < 2 % of TCO. Switching the chat path to a PTU pool, or routing simple intents to a smaller model via APIM, has roughly 100× more impact than tuning sandbox compute.

21.5 PTU vs PAYG — the deployment-mode trade-off

Hosted Agents scale to zero on the **runtime** side, but the **model deployment** behind them does not always behave the same way:

Mode	Behaviour	Best when
PAYG (Pay-as-you-go)	Per-token pricing; no idle charge on the model deployment itself	Spiky / unpredictable traffic, pilots, internal tools, scale-to-zero alignment with the agent runtime
PTU (Provisioned Throughput Units)	Hourly committed capacity , billed even when agents are idle; predictable latency and throughput	Steady high-volume traffic, latency-critical user-facing flows, workloads with strict SLOs

FinOps consequence: if you run a Hosted Agent that genuinely scales to zero overnight, a PAYG model deployment respects that pattern; a PTU pool keeps billing 24×7 regardless. Mix both — PTU for the predictable peak, PAYG (or APIM-fronted overflow) for the long tail — to get the lowest blended cost. Re-validate every quarter with Azure Cost Management.

22 Fine-Tuning & Custom Models

What you will learn

- When fine-tuning beats prompting + RAG (and when it does not).
- The 5-step Foundry fine-tuning pipeline, with the eval gate that must run before promotion.
- How to pick a model by workload fit (Model Router) and what **Fireworks AI on Foundry** (GA) adds for open-model + BYOM inference.

Industrialization isn't only about wiring agents — sometimes the right move is to specialize a model, route to a different one, or run an open-weight model behind the same Azure endpoint.

22.1 When fine-tuning beats prompting + RAG

Signal	Action
Fixed format / DSL output	Fine-tune (SFT) — saves tokens and is more reliable
Brand-specific tone / style	Fine-tune on a few hundred examples
Knowledge that changes weekly	Stay with RAG (Foundry IQ / Azure AI Search)
Deterministic policy enforcement	Function-calling + Toolbox, <i>not</i> fine-tuning
Reasoning improvement on niche domain	DPO / RFT on o-series (preview)

22.2 How to do it on Foundry

1. **Prepare** a JSONL dataset (system + user + assistant) and validate with the Foundry dataset CLI.
2. **Submit** a fine-tune job from the Foundry portal or via `azure-ai-projects` SDK (`client.fine_tuning.jobs.create(...)`). Supported families today: GPT-4.1-mini SFT, GPT-4o-mini SFT, Llama-3 SFT, several Phi variants.
3. **Deploy** the resulting checkpoint as a regular Foundry deployment — agents reference it like any other model.
4. **Evaluate** the fine-tuned model in the same eval pipeline as the base (see §25) before promotion.
5. **Govern** — fine-tuned weights are tenant-isolated and never leave the Foundry boundary; tag deployments with `model-lineage` in IaC for audit.

Combine fine-tuning with RAG: fine-tune for **format and tone**, retrieve for **facts**. This pattern is used by most production agents at scale.

22.3 Choosing a model — workload fit, not leaderboard rank

Build 2026's guidance is that model selection is a continuous operating discipline (**Select** → **Evaluate** → **Optimize** → **Operate** → **Improve**), not a one-time pick. Define the task contract first (capability, safety, latency, cost), then choose:

Workload need	Favor	Why
Classification, routing, extraction, high-volume chat	Smaller, lower-latency model	Keeps cost and latency low
Complex reasoning, coding, planning	Stronger reasoning model	Quality on harder tasks
Image, speech, voice, physical AI	Modality-specific model	Matches input/output type
Mixed-complexity traffic	Model Router	Routes each request by quality, cost and latency automatically
Domain-specific tone / format	Fine-tuned or custom model	Consistency for your scenario

Model Router (in Foundry Models) lets you skip manual selection: it routes each request to the most appropriate model for that request's complexity, cost target and latency budget. **Instant models** (preview) let you call deployment-less models — including as an evaluation judge — without provisioning a deployment.

22.4 Fireworks AI on Foundry (GA)

Fireworks AI on Foundry is generally available — production-grade open-model inference through a single Azure endpoint, with enterprise SLAs, PTU Data Zone support, SOC 2 readiness, and the same access controls and audit logging that govern Foundry. During preview it processed **>176 billion tokens across 17 S&P 500 enterprises**; production adopters include Perplexity, Motif, UiPath and StackBlitz.

What you get	Detail
Catalog models	DeepSeek (v3.1, V4 Pro), Qwen 3.5 (9B → 397B), Moonshot Kimi (K2, K2 Thinking, K2.6), Zhipu GLM (4.7, 5.1), Google Gemma 4, Meta Llama 3.1, Mistral Ministral 3 — often before they ship directly from Azure
BYOM (custom weights)	Import full-weight open models (Kimi, GLM, OpenAI gpt-oss-120b, Qwen) onto Fireworks GPU infra via azd (CLI-first). LoRA/adapters not supported
Offers	Data Zone Standard (per-token) and Global Provisioned Throughput ; all support the OpenAI /v1 Chat Completions API and the Foundry SDK/Responses API
Regions	Data Zone Standard in East US, East US 2, Central US, North Central US, West US, West US 3; Global PTU in all global regions except Azure Government

□ **Data-privacy caveats — read before regulated use.** Fireworks on Foundry shares data between Microsoft and Fireworks AI. It is **excluded from EU Data Boundary commitments**, has **not** achieved **FedRAMP**, and **PCI DSS is not applicable** (don't use it to

store/process/transmit cardholder data). Standard (per-token) offerings carry a **15-day model-retirement notice**. Evaluate against your compliance requirements first.

23 Enterprise Context — Work IQ, Fabric IQ, Foundry IQ

What you will learn

- How Foundry IQ became the managed, multi-source knowledge platform (GA at Build 2026) and how its MCP server grounds any agent framework.
- How to expose M365 (Work IQ), Fabric (Fabric IQ — Data agents + Ontology, OneLake MCP) and Foundry-curated knowledge to an agent.
- The chain-of-context pattern for a “morning briefing” type agent, end-to-end.

The reference architecture (§9) shows three context layers. Here’s how an agent actually reaches them.

23.1 Foundry IQ at Build 2026 (June 2026)

Foundry IQ knowledge bases and the Foundry IQ MCP server are now GA. A single knowledge base can fan out across **Azure AI Search, Azure Blob Storage, OneLake, Work IQ, Fabric IQ (Data agents + Ontology), File Search, Azure SQL and MCP** sources — with agentic retrieval, document-level security, cross-source ranking and activity logging — and is exposed as a **remote MCP server** consumable from MAF, Claude, ChatGPT, LangChain or any MCP-compatible host. Highlights:

Capability	Status	Note
Knowledge bases + Foundry IQ MCP server	GA	Stable APIs, SLA, compliance certs, network isolation, managed identity
GA knowledge sources	GA	Azure Blob (with indexing-status API), search indexes, Web, OneLake
New knowledge sources	Preview	Work IQ, Fabric IQ (Data agents + Ontology), File Search, Azure SQL, MCP
Foundry IQ Serverless (Developer tier)	Preview	Scale-to-zero RAG, billed in Compute Units (~\$0.24 CU/h); free until billing starts (expected late 2026)
Microsoft Web IQ	Limited access	Web/news/image/video/shopping grounding, sub-165 ms, zero data retention , honors publisher preferences

Capability	Status	Note
Data-pipeline upgrades	Preview	Layout-aware ingestion, image enrichment/verbalization, first-class SharePoint (ASPX + Lists) indexing
Security upgrades	Preview	Cross-tenant CMK via federated identity, Purview sensitivity-label auditing, incremental SharePoint permissions sync. Private connectivity (Shared Private Link, Network Security Perimeter) is GA

Quality: the latest agentic-retrieval engine improves answer quality by **up to 20%** across datasets/effort-tiers/model-sizes, lifts recall **up to 54%** vs single-shot RAG, and applies server-side token caching to spend fewer tokens per multi-turn conversation.

23.2 Foundry IQ — curated knowledge & multi-source agentic retrieval

Create a Foundry IQ knowledge base, attach your sources, and either wire it as a Toolbox tool (as in §7) or point any agent at its **MCP server endpoint** directly. The knowledge base handles planning, multi-source search, cross-source ranking and citations; you no longer build a retriever per source.

```
# Ground any MAF agent on a Foundry IQ knowledge base via its GA MCP server
agent = AzureOpenAIResponsesClient(credential=AzureCliCredential()).as_agent(
    name="PolicyAgent",
    instructions="Answer grounded only in retrieved enterprise knowledge;
    ↪ cite sources.",
    tools=[client.get_toolbox_tool(name="foundry-iq-knowledge")], # KB
    ↪ exposed over MCP
)
```

The same MCP endpoint is reachable from Claude, ChatGPT, LangChain and other MCP hosts — so the knowledge layer is built once and reused across the whole agent fleet.

23.3 Fabric IQ — business data via Microsoft Fabric

Build 2026 update: Fabric IQ is now a native Foundry IQ knowledge source (pre-view) — agents can query **Fabric Data agents** and company **Ontologies** (formal models of business entities, relationships and rules linked to live OneLake data and a semantic layer), returning structured answers *alongside* unstructured document context in one retrieval call. This is in addition to **Fabric Data Agent** and **OneLake MCP** being **GA** (Fab-Con 2026), which expose Lakehouses, KQL databases and semantic models through a managed MCP endpoint. Entra identity and Fabric workspace permissions flow through automatically.

The legacy pattern below — wrapping Fabric items in a custom MCP server you host yourself — remains useful when you need to mix Fabric data with bespoke business logic, but is no longer the **only** (or the simplest) way to give an agent access to OneLake.

Expose Fabric items (semantic models, OneLake tables, KQL DBs) as MCP servers and add them to a toolbox. Use the OneLake / Fabric REST APIs behind the MCP server, with Entra **on-behalf-of** so the user's Fabric permissions are honored.

```
client.beta.toolboxes.add_tool(
    toolbox_name="finance-toolbox",
    tool={
        "type": "mcp",
        "server_label": "fabric-onelake",
        "server_url": "https://fabric-mcp.contoso.com/mcp",
        "project_connection_id": "<FABRIC_OBO_CONN>"
    },
)
```

23.4 Work IQ — Microsoft 365 graph (mail, files, chats, meetings)

Build 2026 update: Work IQ is now a native Foundry IQ knowledge source (pre-view) — it brings organizational signals (emails, meetings, files, Teams messages) into one enriched, AI-ready source **while respecting user permissions**, so agents can answer “what was decided / what’s top of mind” without a bespoke connector.

When you need finer control, the classic path still works: call the **Microsoft 365 Copilot APIs** (Work IQ retrieval and Search APIs) inside a thin MCP server that authenticates via Entra OBO. Either way, the agent only ever sees content the calling user could see in M365.

```
agent = Agent(
    client=client,
    name="MeetingPrepAgent",
    instructions="Summarize tomorrow's meetings and pull related docs from
    ↪ M365.",
    tools=[client.get_toolbox_tool(name="m365-workiq-toolbox")],
)
```

23.5 Pattern: chain Foundry IQ → Fabric IQ → Work IQ in a single agent

A “morning briefing” agent typically (1) retrieves curated playbooks from **Foundry IQ**, (2) joins KPIs from **Fabric IQ**, and (3) personalizes with **Work IQ** (calendar, recent mail). With Build 2026 you can collapse all three into a **single multi-source Foundry IQ knowledge base** for a unified, permission-aware retrieval call; keep them as separate Connected Agents (A2A) when each layer has its own owner team and lifecycle.

24 Compliance, Data Residency & Sovereignty

What you will learn

- The Foundry mechanism for each typical compliance requirement (GDPR, residency, PII, audit, certifications, sovereign clouds).
- Why a per-agent DPIA is non-optional and what it must contain.

Production agents handle sensitive data — design for the regulatory perimeter from day one.

Requirement	Foundry mechanism
GDPR / EU Data Boundary	Deploy Foundry projects in EU regions; enable EU Data Boundary on the parent subscription; Microsoft 365 Copilot data stays in the boundary by default
Data residency for memory	Memory is co-located with the project region; verify in Project → Settings → Region
PII redaction	Use Azure AI Content Safety / Prompt Shields for harmful content and prompt-injection protection; use Azure Language PII detection (or an equivalent DLP tool) for PII identification, classification and redaction. Apply both at APIM ingress <i>and</i> in a Toolbox pre-tool policy
Right to erasure	Use the Memory CRUD API to surface and delete user-scoped memories on request
Audit & traceability	OpenTelemetry traces → Log Analytics with 1+ year retention; Entra Agent ID logs → Microsoft Purview
Model provenance	Tag every deployment with <code>model.family</code> , <code>model.version</code> , <code>data.classification</code> in IaC; enforce via Azure Policy
Certifications	Foundry inherits Azure compliance offerings — ISO 27001/27017/27018, SOC 1/2/3, HIPAA, FedRAMP High (regional). See the Microsoft Trust Center
Sovereign clouds	Foundry availability in Azure Government / Azure China is on the roadmap — check the trust portal before committing

Document the data flow in a one-page DPIA per agent: who is the data subject, where the data is stored, retention, model used, transfer mechanism. This is the artifact your DPO will ask for.

25 Advanced Quality Evaluation — Coherence, Groundedness, Hallucination

What you will learn

- The built-in evaluators in the Azure AI Evaluation SDK and which ones must gate every promotion.
- The four evaluation targets in the Foundry portal and how to validate with your own data.
- How to wire a continuous eval into CI and grow the golden dataset from production traces.

Section §15 covered chaos and resilience tests. This section adds **semantic** evaluation — the only way to catch silent regressions in an LLM-driven system. Build 2026's core message: **benchmarks are not enough — validate against your own prompts, data, users and business rules**, and re-run continuously as model versions, fine-tuned variants and new model families arrive.

25.1 Evaluate in the Foundry portal — four targets

The portal ([Evaluation quickstart](#)) runs your model/agent against test data and scores it with built-in or custom evaluators. Pick a target:

Target	Evaluates	Best for
Agent	Output of a prompt or hosted agent	End-to-end behavior — Full conversations (preview) or Individual turns
Model	A deployed (or instant) model's completions	Benchmarking candidates against a curated set
Dataset	Pre-existing outputs in CSV/JSONL	Scoring outputs without re-running the model/agent
Traces	Agent interactions captured in App Insights	Evaluating historical production traces over a time range

- **Bring your own data:** CSV/JSONL with prompts, expected outputs, labels or ground-truth answers; run side-by-side comparisons across models and prompts.
- **Simulated data:** generate synthetic multi-turn conversations from scenario descriptions to test before deployment.
- **Instant models** can serve as either the evaluation target *or* the judge model, selected straight from the model picker (no deployment needed).
- An Azure OpenAI connection with a deployed judge model (e.g. gpt-4.1-mini) is required for AI-assisted quality evaluators; you need the **Foundry User** role on the project.

25.2 Built-in evaluators (Azure AI Evaluation SDK)

Evaluator	What it scores	Use in pipeline
RelevanceEvaluator	Answer addresses the question	PR gate, threshold $\geq 4/5$
CoherenceEvaluator	Logical flow & readability	PR gate
GroundednessEvaluator	Answer derives from retrieved context (anti-hallucination)	PR gate, threshold $\geq 4/5$
FluencyEvaluator	Linguistic quality	Optional
SimilarityEvaluator	Match against a golden answer	Regression suite
ContentSafetyEvaluator	Hate, sexual, violence, self-harm	Mandatory gate
ProtectedMaterialEvaluator	Copyright leakage	Mandatory gate for public-facing agents
IndirectAttackEvaluator	Prompt injection robustness	Run after every Toolbox change

25.3 Example — wire a continuous eval into CI

```

from azure.ai.evaluation import evaluate, GroundednessEvaluator,
    ↪ RelevanceEvaluator
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential

project = AIProjectClient(endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
                           credential=DefaultAzureCredential())

results = evaluate(
    data="eval/golden_dataset.jsonl",          # 50-500 curated Q/A/context rows
    target=lambda row: agent.run(row["question"]),
    evaluators={
        "relevance": RelevanceEvaluator(model_config=
            ↪ project.connections.get_default_llm()),
        "groundedness": GroundednessEvaluator(model_config=
            ↪ project.connections.get_default_llm()),
    },
    output_path="eval/results.json",
)
assert results["metrics"]["groundedness.mean"] >= 4.0, "Groundedness
    ↪ regression — block deploy"
# NOTE: the exact key path and scale depend on the azure-ai-evaluation SDK
    ↪ version
# (some releases nest under results["metrics"]["groundedness"]["mean"],
    ↪ others use
# 1-5 instead of 0-1). Pin the SDK version in requirements.txt and adapt the
# assertion to the structure your version returns.

```

25.4 Red Team Agent (preview)

Schedule the **AI Red Teaming Agent** weekly against your stable endpoint to surface jailbreaks, indirect injection vectors, and unsafe completions before customers do.

25.5 Build your golden dataset from production traces

Curate 50–200 high-value traces per release from App Insights → label them in the Foundry data labeling UI → use as the regression baseline. Refresh quarterly.

26 Migration Paths & .NET Parity

What you will learn

- Concept-level mapping from Semantic Kernel and AutoGen to MAF v1.0.
- The .NET parity story and the lifecycle policy for SDK and runtime upgrades.

26.1 From Semantic Kernel to MAF v1.0

SK concept	MAF v1.0 equivalent
Kernel + KernelPlugin	Agent + Tool
IChatCompletionService	ChatClient (Azure OpenAI / Anthropic / etc.)
KernelFunction decorator	@tool decorator
Planners (Sequential/Stepwise)	Workflow with sequential / hand-off patterns
Memory plugin	FoundryMemoryProvider / InMemoryHistoryProvider
Filters / Hooks	Middleware pipeline

Use the [official MAF migration guide](#) — most ports are mechanical and can be done one agent at a time, while keeping the rest of the SK app untouched.

26.2 From AutoGen to MAF v1.0

AutoGen's GroupChat and Society of Mind map directly to MAF **multi-agent workflows** (concurrent / hand-off / sequential). Conversation state is preserved via the InMemoryHistoryProvider; long-running orchestrations move to CompactionProvider + Memory for persistence.

26.3 .NET parity

MAF v1.0 ships with full feature parity in .NET (NuGet Microsoft.Agents.AI) — same agent / workflow / tool / middleware concepts. The differences are idiomatic, not functional:

```
var client = new AzureOpenAIClient(endpoint, new AzureCliCredential());
var agent = client.AsAgent(
    name: "SupportTriage",
    instructions: "You are an expert triage agent.");

var run = await agent.RunAsync("Classify this ticket: ...");
Console.WriteLine(run.Text);
```

Tooling: hosted-agent deployment via azd is identical for .NET projects (just a different Dockerfile base image). The Foundry Toolkit for VS Code supports both runtimes.

26.4 Update / lifecycle policy

- **Preview → GA window:** typically 3–6 months; subscribe to the [Foundry release notes RSS](#).
- **MAF SDK** follows semver — pin the minor version in `pyproject.toml` / `Directory.Packages.props` and update via Dependabot.
- **Hosted-agent runtime image** is patched by Microsoft; rebuild and redeploy your container monthly to pick up base-image CVE fixes.
- **Deprecation policy:** Azure AI services give 12 months notice before retiring a model deployment. Track via Azure Service Health alerts.

27 Lifecycle Management — Versioning, Rollback, Decommissioning

What you will learn

- The five versioning axes (code, declaration, model, tool, memory schema) and the gate for each.
- Rollouts, rollback, the model-upgrade discipline, and the decommissioning checklist.

A production agent is a **long-lived asset**, not a one-shot deployment. Foundry treats every promoted agent as a versioned resource — and so should you.

27.1 Versioning model

Version axis	Mechanism	Promotion gate
Agent code	Container image tag agent:<git-sha> pushed to ACR by CI	Build green + unit tests + plugin tests
Agent declaration	agent.yaml revision (instructions, model, tools, memory cfg)	Diff review; no model downgrade without sign-off
Underlying model	Pinned model deployment name + version (e.g. gpt-4.1-2025-04-14)	Eval suite (\$25) + Red-Team Agent regression must not regress

Version axis	Mechanism	Promotion gate
Tool / Toolbox version	Tool manifest version field; Toolbox honors per-agent pin	Contract tests on tool input/output schemas
Memory schema	Long-term memory schema_version in metadata	Backward-compat check before any breaking change

27.2 Rollouts and rollback

Foundry exposes **stable agent endpoints** with weighted routing across versions — the same primitive used by Azure App Service slots, applied to agents.

YAML below is illustrative — not a stable schema. Validate the exact field names against the installed `azure.ai.agents` extension version. The capability (immutable versions + weighted traffic routing + auto-rollback) is documented; the precise YAML keys may differ from the example.

```
# excerpt of agent.yaml
deployment:
  strategy: weighted
  versions:
    - tag: v1.4.2    # stable, current production
      weight: 90
    - tag: v1.5.0    # canary
      weight: 10
rollback:
  on_eval_regression: true          # auto-rollback if eval gate fails
  ↪ post-promotion
  on_error_rate_above: 0.02        # 2% over 5 min
  retain_n_previous: 5             # keep last 5 versions warm for instant
  ↪ rollback
```

Rollback is a **traffic-weight change** — no redeployment, no cold start.

27.3 Continuous improvement loop

Treat production as an evaluation set:

1. **Sample** 1–5% of production conversations daily into Foundry **Tracing UI**
2. **Auto-label** with built-in evaluators (groundedness, coherence, safety) and human SME review for the long-tail
3. Failed traces feed a **regression dataset** — appended to the eval suite so the bug never returns
4. Retrain prompts / fine-tune (§22) on the curated set when patterns emerge
5. Promote through the same `azd` + canary pipeline → loop back to step 1

27.4 Model upgrade strategy

Microsoft **announces model deprecations** on Azure with a 6-month advance notice (typical). Build the upgrade muscle:

- Schedule a **quarterly model sweep** — re-run the eval suite against newer model versions of the same family (e.g., gpt-4.1 → gpt-4.5 → gpt-5)
- Track **cost-quality trade-offs** in a comparison report — sometimes a smaller / cheaper model wins after prompt-tuning
- Prefer **deployment aliases** (prod-chat-model) over hardcoded version strings in agent.yaml so swap = config change

27.5 Decommissioning checklist

Retiring an agent is a documented event, not a delete-and-forget:

1. Announce **end-of-life date** to consumers (Teams notice, M365 admin center, in-channel deprecation banner)
2. Switch the stable endpoint to **read-only mode** (returns a polite “this agent is retired — use X instead” message)
3. Drain weighted traffic to **0%** over the grace period (typically 30–90 days)
4. **Snapshot** Memory store, traces, and audit logs to long-term archive (immutable storage) per data-retention policy
5. **Revoke** the Entra Agent ID and all RBAC role assignments
6. **Remove** Toolbox bindings and Logic Apps connections
7. **Decommission** ACR image tags after the audit-retention window (don’t delete sooner — you may need to re-deploy for forensic replay)
8. Update the **agent registry / Agent 365 catalog** (§28) to mark the agent **Retired** and document the replacement

28 Agent 365 & Enterprise Control Plane

What you will learn

- When Agent 365 starts to matter (≥ 4 –20 agents in a portfolio).
- The capabilities it brings beyond a single Foundry project — registry, identity governance, Conditional Access, RAI gates, chargeback.

Status: Announcement / early availability — feature scope and naming may evolve.

Production readiness: Use selectively for fleet-level governance; not required for individual agents.

When to think about Agent 365: not on day one. A single agent can ship through Foundry alone. Once you operate **a fleet** — multiple business units, multiple Foundry projects, mixed Copilot Studio + custom agents — the per-project model gets noisy and you need a portfolio control plane. **Agent 365** is Microsoft’s emerging admin layer for that case; it is *not* a mandatory dependency for building one or two agents.

28.1 What Agent 365 provides

Capability	What it gives the IT / security org
Centralized agent registry	Single inventory of every agent across every Foundry project + Copilot Studio + Microsoft 365 — owners, business unit, data classifications, certification status
Identity governance for agents	Lifecycle of Entra Agent IDs as a managed resource — provisioning, attestation, revocation; integrates with Entra Privileged Identity Management
Conditional Access for agents	Apply Entra Conditional Access policies to agent invocations — restrict by location, device compliance, risk score, MFA-on-behalf-of-user
Usage policies & approvals	Centrally enforce: human-in-the-loop required for X tool families; agent allowed to spend $\leq Y$ tokens/day; agent can or cannot be invoked from public channels
Cost & quota at portfolio level	Roll-up of Foundry project costs by business unit, with chargeback metadata
Data residency & DLP enforcement	Project-level constraints (region, allowed connectors, Purview labels) enforced before deployment
Cross-agent observability	Aggregate dashboards: failure-rate heatmap, top-N expensive agents, regression alerts, compliance drift
Responsible AI evidence collection	Automated capture of eval reports, Red-Team results, content-safety incidents — exportable for regulator / audit committee

28.2 Governance with Azure Policy

Beyond Agent 365's policy engine, **Azure Policy** rules can be attached to the resource group hosting Foundry projects to enforce:

- Disallow agent deployment if **no keyvault_connection** is declared
- Require **tags.costCenter** and **tags.dataClassification** on every Foundry project
- Block deployment in non-approved **regions** (data residency)
- Require **OpenTelemetry export** to the central Log Analytics workspace
- Require minimum **Content Safety policy strictness** per project

28.3 Responsible AI gating

For agents touching regulated data or interacting with end customers, Microsoft recommends an internal **Responsible AI Impact Assessment** (template available from Microsoft) covering: intended use, harms taxonomy, fairness, transparency, accountability, human oversight. Agent 365 lets you **block deployment** until the RAI assessment is signed off and attached to the agent record.

28.4 Adoption pattern

A simple maturity ladder is enough — start with **per-project governance** at 1–3 agents; **adopt the Agent 365 registry** when you cross 4–20 agents; layer **Conditional Access, RAI gates, and chargeback** once you operate 20+ agents across business units.

The earlier you onboard Agent 365, the cheaper compliance becomes — retro-tagging an estate of 50 agents is painful.

29 References & Further Reading

29.1 Get started

- [Microsoft Foundry portal](#)
- [Microsoft Foundry documentation](#)
- [Develop generative AI apps — Microsoft Learn training path](#)

29.2 Source articles (this guide synthesizes and extends them)

- [A Developer's Guide to Managing Models, Cost and Quality in Microsoft Foundry \(Build 2026\)](#)
- [Build smarter agents faster with Foundry IQ \(Build 2026\)](#)
- [Introducing the new Hosted Agents in Foundry Agent Service](#)
- [Introducing Toolboxes in Foundry](#)
- [Deploying Production-Grade Agents on Microsoft Foundry — end-to-end developer journey](#)
- [Microsoft Agent Framework v1.0 announcement](#)
- [Agent Harness in Microsoft Agent Framework](#)
- [Build AI agents with GitHub Copilot SDK and Microsoft Agent Framework](#)
- [Microsoft Foundry Toolkit for VS Code is now Generally Available](#)

29.3 Microsoft Learn

- [Hosted agents — concept docs](#)
- [Hosted agent quickstart with azd](#)
- [Toolbox in Foundry — how-to](#)
- [Memory in Foundry Agent Service](#)
- [Microsoft Agent Framework documentation](#)
- [Set up a Key Vault connection in Foundry](#)
- [Foundry Agent Service limits, quotas & regions](#)
- [Foundry region availability](#)
- [Foundry what's new / release notes](#)
- [Fireworks AI on Foundry — enable & deploy models](#)
- [Evaluate a generative AI app in the Foundry portal \(quickstart\)](#)
- [Manage and increase quotas for resources](#)
- [Connect an agent to Foundry IQ knowledge bases](#)
- [Fine-tuning models on Azure AI Foundry](#)
- [Azure AI Evaluation SDK](#)
- [AI Red Teaming Agent](#)

- [MAF migration guide \(Semantic Kernel & AutoGen → MAF v1.0\)](#)
- [Microsoft Agent Framework for .NET](#)
- [EU Data Boundary for Microsoft Cloud services](#)
- [Azure SLA — AI Foundry](#)
- [Azure pricing — AI Foundry](#)

29.4 Tooling

- [azd \(Azure Developer CLI\) for Foundry hosted agents](#)
- [Foundry Toolkit for VS Code](#)
- Sample code — Toolbox + Hosted Agents:
 - [Microsoft Agent Framework](#)
 - [LangGraph](#)
 - [Copilot SDK](#)

29.5 Build 2026 sessions & skilling

- [BRK230 — Build smarter AI systems in Foundry as models and costs evolve](#)
- [BRK246 — Foundry IQ: Fuel agents with enterprise knowledge and agentic retrieval](#)
- [BRK240 — Build context-aware agents: From data to decisions with Microsoft IQ](#)
- [LAB532 — From data to context: agent-ready knowledge with Foundry IQ](#)
- [DEM331 — Turn APIs, tools, and data into real agent velocity](#)
- [Claude in Microsoft Foundry — Skilling Learning Path](#)
- [Mastering Foundry IQ — Forgebook cookbook](#)

29.6 Community

- [Foundry Discord](#)
- [Foundry GitHub forum](#)
- [Microsoft Mechanics — Hosted Agents \(12-min video\)](#)
- [Live series — *Host your agents on Foundry*](#)